Thesis of the end-of-study project

To obtain the State Engineering Diploma

Major: Computer Engineering

STM32Cube Competitor Benchmark

Authored by :

Fares Dkhili

Defense scheduled for September 2026 before the following jury:

Mrs. Anissa Omrane	FST	- President
Mrs. Sana Younes	FST	- Examiner
Mr. Aymen Zemrani	FST	- Academic supervisor
Mrs. Sirine Eljazi	STMicroelectronics	- Professional supervisor

Class of : 2025/2026

Dedicated to

To my dear parents, whose unconditional love, patience, and countless sacrifices have carried me to this moment. Everything I have achieved is a reflection of the values you taught me.

To my family and my friends, who never stopped believing in me and who lifted me through every doubt and every long night. This work is as much yours as it is mine.

Fares Dkhili.

Acknowledgments

This project marks the end of my engineering studies, and it could not have come together without the people who guided and supported me along the way.

I am deeply grateful to Ms. Sirine Eljazi, my supervisor at STMicroelectronics, for her trust, her availability, and the technical insight she shared throughout this work. Her guidance shaped both the project and the engineer I have become. I also thank the entire STMicroelectronics team for the warm welcome and for making my time within the company a genuine learning experience.

I would like to express my sincere thanks to Mr. Aymen Zemrani, my academic supervisor at the Faculty of Sciences of Tunis (FST), for his rigorous follow-up, his sound advice, and his constant encouragement from the first day to the last.

My thanks also go to the members of the jury: to Ms. Anissa Omrane for the honor of chairing the jury, and to Ms. Sana Younes for accepting to examine and evaluate this work. Their time and attention are sincerely appreciated.

I extend my gratitude to the teaching staff of the Faculty of Sciences of Tunis at the University of Tunis El Manar, whose dedication over these years laid the foundations on which this project was built.

Finally, my heartfelt thanks to my family and friends, whose unwavering support and patience accompanied me through every stage of this journey.

Résumé

Ce rapport présente un projet de fin d'études réalisé au sein de STMicroelectronics Tunis afin de comparer l'écosystème STM32Cube à des écosystèmes concurrents au moyen de scénarios logiciels contrôlés. Nous avons établi une méthode de benchmarking fondée sur des fonctionnalités équivalentes, des conditions de compilation documentées, l'empreinte mémoire extraite des fichiers linker map, des mesures d'exécution lorsqu'elles étaient étayées par des observations matérielles, ainsi qu'une vérification explicite de l'équivalence sémantique et de l'équité des comparaisons. Nous avons également conçu deux outils de support : *MCU Workflow Studio*, qui structure l'analyse inter-vendeurs des fichiers linker map, et le *MCU Benchmark Copilot Agent*, qui formalise le processus de création, de portage, de compilation, de traçabilité et de validation des projets.

La campagne ST–GD32 achevée couvre la pile USB device à travers les scénarios Human Interface Device (HID) et Communication Device Class (CDC), ainsi que FreeRTOS à travers des scénarios d'ordonnancement basique, coopératif et préemptif. STM32 nécessite 35–88 % de mémoire flash supplémentaire selon les scénarios, principalement en raison de la couche Hardware Abstraction Layer (HAL), mais utilise 57–65 % de RAM en moins dans les scénarios USB. Dans les observations rapportées, l'initialisation USB est 61,7 % plus rapide sur STM32, les mesures USB en régime établi sont proches et les mesures FreeRTOS diffèrent au maximum de 3,3 %. Ces résultats nous ont conduits à retenir le module Reset and Clock Control (RCC) de la HAL et la sélection des fonctionnalités USB à la compilation comme pistes d'investigation ciblées.

La comparaison ST–Texas Instruments associe une analyse statique des pilotes Low-Layer de STM32CubeC5 et de la bibliothèque de pilotes MSPM33 à douze projets bare-metal appariés couvrant la conversion analogique-numérique, les entrées-sorties généralistes, l'I^2C, le SPI et l'UART. Sur l'ensemble de ces douze images liées indépendamment, STM32C5 utilise 2,8 % de mémoire flash agrégée supplémentaire et 5,0 % de RAM agrégée en moins. Ces résultats caractérisent la structure du code, la couverture des Software Development Kits (SDK) et l'empreinte des images complètes. Nous avons ensuite comparé FreeRTOS à travers des scénarios basique, coopératif et préemptif. Les trois images STM32C5 utilisent 27,9–29,6 % de flash en moins, tandis que MSPM33 économise 56 à 76 octets de RAM. Des observations d'exécution ont également été collectées, mais STM32C5 fonctionne à 144 MHz contre 32 MHz pour MSPM33. Après normalisation par la fréquence, le scénario

...

basique devient presque équivalent; les scénarios coopératif et préemptif restent affectés par la configuration du noyau et par des primitives de synchronisation différentes. Ces limites empêchent de conclure globalement sur l'efficacité de l'ordonnanceur.

Mots clés : STM32Cube, benchmarking, empreinte firmware, microcontrôleur, FreeRTOS, USB, GD32, Texas Instruments, pilotes bas niveau, comparaison inter-vendeurs.

Summary

This report presents an end-of-studies project carried out at STMicroelectronics Tunis to compare the STM32Cube ecosystem with competing microcontroller ecosystems through controlled software scenarios. We established a benchmarking method based on matched functionality, documented build conditions, firmware footprint extracted from linker maps, runtime measurements where hardware evidence was available, and an explicit review of semantic equivalence and fairness. We also designed two supporting tools: *MCU Workflow Studio*, which structures cross-vendor linker-map analysis, and the *MCU Benchmark Copilot Agent*, which encodes the project-creation, porting, build, provenance, and validation workflow.

The completed ST–GD32 campaign covered the USB device stack through Human Interface Device (HID) and Communication Device Class (CDC) scenarios, and FreeRTOS through basic, cooperative, and preemptive scheduling scenarios. STM32 required 35–88 % more flash across these scenarios, mainly because of the Hardware Abstraction Layer (HAL), while it used 57–65 % less RAM in the USB scenarios. USB initialisation was 61.7 % faster on STM32 in the reported observations, steady-state USB measurements were close, and the FreeRTOS measurements differed by at most 3.3 %. These results led us to identify the HAL Reset and Clock Control (RCC) module and USB compile-time feature selection as targeted areas for investigation.

The ST–Texas Instruments comparison combined static analysis of the STM32CubeC5 Low-Layer drivers and the MSPM33 driver library with twelve matched bare-metal projects covering analog-to-digital conversion, general-purpose input/output, I^2C, SPI, and UART. Across these twelve independently linked images, STM32C5 used 2.8 % more aggregate flash and 5.0 % less aggregate RAM. We then compared FreeRTOS through basic, cooperative, and preemptive scenarios. The three STM32C5 images used 27.9–29.6 % less flash, while MSPM33 used 56–76 fewer RAM bytes. Runtime observations were also collected, but STM32C5 ran at 144 MHz and MSPM33 at 32 MHz. The basic result became nearly equal after clock normalisation; the cooperative and preemptive results remained affected by kernel configuration and non-equivalent synchronisation primitives. These constraints prevent a general scheduler-efficiency conclusion from the current measurements.

Key Words: STM32Cube, benchmarking, firmware footprint, microcontroller, FreeRTOS, USB,

GD32, Texas Instruments, low-level drivers, cross-vendor comparison.

ملخص

يقدّم هذا التقرير مشروع نهاية الدراسات المنجز بشركة إس تي مايكروإلكترونيكس بتونس بهدف مقارنة منظومة STM32Cube بمنظومات متحكمات دقيقة منافسة من خلال سيناريوهات برمجية مضبوطة. اعتمدنا وظائف متكافئة وشروط بناء موثقة، واستخرجنا البصمة الذاكرية من ملفات الربط، كما راجعنا التكافؤ الدلالي وعدالة المقارنة.

طوّرنا أداتين مساعدتين: MCU Workflow Studio لتنظيم تحليل ملفات الربط بين المصنّعين، و MCU Benchmark Copilot Agent لتنظيم إنشاء المشاريع ونقلها وبنائها وتتبع مصادرها والتحقق منها. تساعد الأداتان على توثيق سير العمل، لكن صحة كل نتيجة تبقى مرتبطة بأدلة البناء والقياس والعتاد.

غطّت مقارنة ST–GD32 سيناريوهات USB HID و CDC وثلاثة سيناريوهات لـ FreeRTOS . استهلك متحكم إس تي ذاكرة فلاش أكثر بنسبة ٨٨--٣٥ %، واستخدم ذاكرة RAM أقل بنسبة ٦٥--٥٧ % في سيناريوهات USB . أظهرت القياسات المسجلة فرقاً قدره ٦١٫٧ % في زمن تهيئة USB لصالح إس تي، غير أن غياب العينات المتكررة وحدود القياس المفصلة يمنع تعميم تفسير سببي.

جمعت مقارنة ST–Texas Instruments بين تحليل ثابت للمشغلات واثني عشر مشروعاً متكافئاً. استخدم متحكم إس تي فلاشاً إجمالياً أكثر بنسبة ٢٫٨ % وذاكرة RAM أقل بنسبة ٥٫٠ %. لا تسمح الأدلة الحالية باستنتاجات حول زمن التنفيذ أو الطاقة قبل توحيد الساعات وإعدادات الوحدات وحدود القياس وبروتوكول أخذ العينات.

الكلمات المفتاحية:

STM32Cube ، قياس الأداء، البصمة الذاكرية، متحكم دقيق، FreeRTOS ، USB ، GD32 ، Texas Instruments ، مقارنة بين المصنّعين.

List of Figures

List of Tables

Listings

Contents

Chapter 1

General Description of the Project

Introduction

This chapter introduces the host company and the context of the project. The existing problems are detailed to define the specifications and clarify the requested work. We then present the theoretical foundations relevant to the project and explain the adopted work methodology.

1.1 Host company

This section provides an overview of STMicroelectronics, its history, the ST Tunis site, and the technically oriented marketing and application support team in which this project was carried out.

1.1.1 Presentation

STMicroelectronics is a global semiconductor company dedicated to enhancing people's lives through innovative electronic solutions [1]. With one of the most extensive product portfolios in the industry, ST serves over 200,000 customers worldwide, achieving net revenues of $13.27 billion in 2024 [2].

The company was established in June 1987 as SGS-THOMSON Microelectronics, following the merger of SGS Microelettronica (Italy) and Thomson Semiconductors (France). The name "STMicroelectronics" was officially adopted in May 1998. Figure 1.1 shows the company's official logo.

Figure 1.1: STMicroelectronics logo.

With more than 80 sales and marketing offices and 14 primary manufacturing locations across the globe, the corporation employs over 50,000 people [1].

1.1.2 STMicroelectronics vision and markets

STMicroelectronics has prioritised technological innovation in its strategy for over 25 years. It makes market-driven investments in technology development with the goal of commercialising cutting-edge innovations that create immediate value for its customers [3]. Today, with a robust pipeline, ST delivers products that are smaller, faster, more energy-efficient, and equipped with new features. Driven by a global lean manufacturing culture, ST addresses three main end markets: automotive, industrial, and personal electronics — and offers engagement models ranging from application-specific embedded systems to application-specific standard products.

1.1.3 STMicroelectronics Tunis

The ST Tunis site, located in Technopark El Ghazela, began operations in 2001 with nine engineers. Today it has expanded to more than 200 employees working across design, tools, applications, quality, and support functions [1]. The facility participates in every stage from the initial design of new circuits to the final verification of their functionality before mass production.

The Microcontroller Division (MCD) is dedicated to the design of microcontrollers, demonstration platforms, and solutions for consumer use. It is responsible for the development of drivers and embedded software — specifically the validation and development of libraries and firmware that drive ST microcontrollers. Figure 1.2 shows the organisational structure of the MCD division at the Tunis site; the Maintenance team, highlighted, is where this project was carried out.

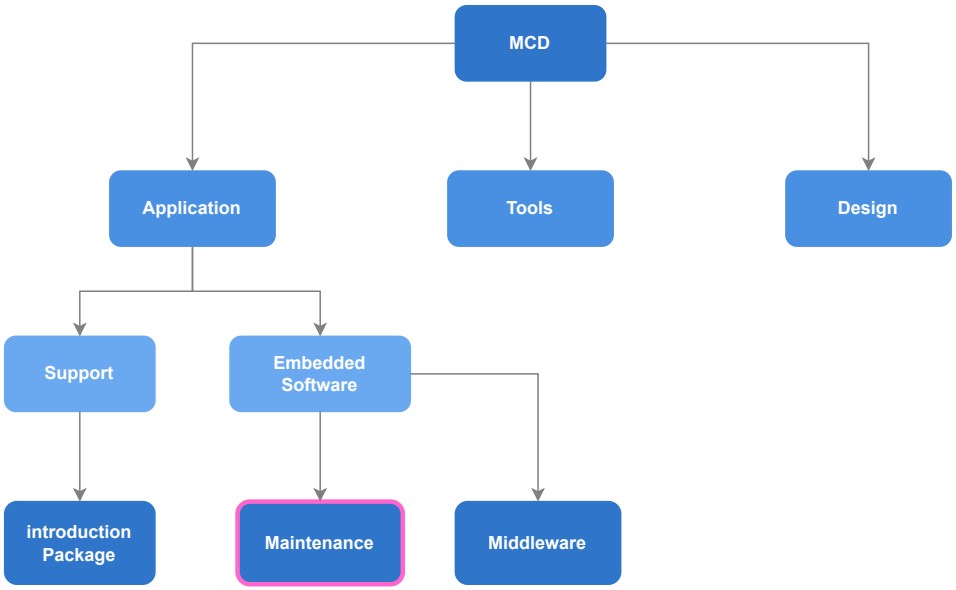

Figure 1.2: Organisational structure of the Microcontroller Division (MCD) at ST Tunis. The Maintenance team (highlighted) is the host team of this project.

This project was carried out in the **Technical Marketing and Application Support** team under MCD. This team includes two sub-teams:

- **The GitHub team**: supports customers via GitHub, publishes updates of HAL drivers, and maintains example projects to facilitate application development based on STM32 microcontrollers.
- **The Maintenance and Validation team**: ensures customer satisfaction through bug maintenance and functional validation of manufactured microcontrollers and their associated HAL versions. This team represents the junction linking software and hardware design processes to implementation and industrialisation.

1.2 Project presentation

1.2.1 Project framework

As part of our training in Computer Engineering at the Faculty of Sciences of Tunis (FST), University of Tunis El Manar (UTM), we conclude our education with an end-of-studies project (PFE). In this context, our project — entitled *STM32Cube Competitor Benchmark* — was scheduled at STMicroelectronics Tunis from February to September 2026. At the date of writing, 3 August 2026, this project window was still in progress.

1.2.2 Project context

The embedded microcontroller market is highly competitive. When an engineer evaluates an MCU for a new design, the choice between STMicroelectronics and competing vendors (GigaDevice GD32, NXP, Texas Instruments, Renesas) often depends on measurable differences: firmware footprint, execution speed, middleware maturity, and development-tool productivity. The technical marketing team at ST needs objective, reproducible data that quantifies where the STM32Cube ecosystem outperforms its competitors, where gaps exist, and what concrete enhancements would close those gaps.

At the start of this project, no systematic benchmarking methodology existed within the team. Cross-vendor comparisons were ad-hoc, performed manually on a case-by-case basis, and difficult to reproduce or communicate convincingly to stakeholders.

1.2.3 Problematic

Producing actionable competitive intelligence for the marketing team requires answering precise questions: for a given software stack (e.g. USB device, FreeRTOS integration), running identical scenarios on ST and on a competitor, how do the two platforms compare in flash and RAM footprint, execution performance, and middleware completeness? And where ST lags, what specific enhancements — at the HAL, middleware, or configuration level — would close the gap?

Answering these questions rigorously raises several challenges:

1. **Scenario design**: each benchmark must exercise the same functionality on both platforms under identical, well-defined conditions so that the comparison is fair and the results are meaningful.

2. **Porting fidelity**: reproducing an STM32 reference scenario on a competitor requires preserving exact runtime semantics — task structure, timing, synchronisation, reporting format — to avoid comparing apples to oranges.

3. **Result extraction and interpretation**: raw numbers (kilobytes, cycles, milliseconds) must be attributed to comparable architectural layers and interpreted in terms of engineering trade-offs, not merely listed.

4. **Scale and reproducibility**: the team needs to cover multiple vendors and multiple stacks with consistent methodology, so that results are comparable across campaigns and over time.

1.2.4 Proposed solution

Our solution is a structured benchmarking methodology centred on three deliverables:

1. **Benchmark scenarios**: for each software stack under evaluation (USB device stack, FreeRTOS, etc.), we design a set of concrete, reproducible scenarios that both the STM32 reference and the competing platform must run identically. Each scenario exercises a specific aspect of the stack (e.g. HID, CDC, and MSC for USB device).

2. **Comparative results and interpretation**: for each scenario, we extract per-layer footprint figures, execution metrics, and qualitative observations, then interpret them to show where ST is stronger, where the competitor leads, and why.

3. **Enhancement proposals**: where the data reveals a gap unfavourable to ST, we identify the root cause and propose targeted improvements — whether at the HAL driver level, in middleware configuration, or in linker/startup overhead.

To support this workflow at scale and ensure reproducibility, we developed two supportive tools in parallel:

- **MCU Workflow Studio** — a cross-vendor firmware footprint analysis application that automates linker-map comparison, layer classification, and result export (detailed in Chapter 2).

- **MCU Benchmark Copilot Agent** — a custom AI agent system that accelerates benchmark creation, porting, and debugging while enforcing fairness and semantic equivalence (detailed in Chapter 3).

These tools are parallel engineering contributions that enhance the benchmarking effort; they are not the benchmarks themselves. The core deliverable remains the benchmark results and the enhancement proposals they yield.

1.2.5 Mission and objectives

The mission of this internship is to provide the STMicroelectronics technical marketing team with rigorous, reproducible, and interpretable competitive benchmark data. Specifically, the project aims to:

1. Design benchmark scenarios for selected software stacks, ensuring fairness and semantic equivalence between ST and each competitor.

2. Execute the benchmarks on representative hardware, extracting flash/RAM footprint, execution performance, and qualitative observations per scenario.

3. Compare and interpret the results layer by layer, highlighting where ST excels and where gaps exist.

4. Formulate concrete enhancement proposals for STM32Cube where the data reveals room for improvement.

5. Develop supportive tools (footprint analyser, benchmark agent) to make the methodology reproducible and scalable beyond this project.

1.3 Theoretical foundations

This section presents the background concepts essential for understanding the technical aspects of the project: the STM32 microcontroller ecosystem, firmware memory footprint, and the linker-map artefact that our tools consume.

1.3.1 STM32 microcontrollers

1.3.1.1 Overview

A microcontroller unit (MCU) is an integrated circuit that combines a processor core, memory (flash and RAM), and peripherals on a single chip. It is designed for embedded applications where cost, power, and real-time determinism are critical constraints.

1.3.1.2 The STM32 family

The STM32 family from STMicroelectronics is a series of 32-bit microcontrollers based on ARM Cortex-M processor cores, produced since 2007 [4]. The family spans four macro groups — high-performance, mainstream, ultra-low-power, and wireless — with more than ten product sub-families. Each sub-family targets a different balance of processing power, memory, peripherals, and energy efficiency.

The ARM Cortex-M profile is optimised for embedded applications requiring low cost and low power consumption [5]. Cortex-M cores range from the M0 (simplest, lowest gate count) to the M7 and M33 (highest performance, with DSP and optional floating-point units). STM32 devices span this entire range.

1.3.1.3 The STM32Cube ecosystem

STMicroelectronics provides a comprehensive software ecosystem around its STM32 hardware [6]:

- **STM32CubeMX**: a graphical configuration tool that generates initialisation code for peripherals, clocks, and middleware.
- **STM32CubeIDE**: an integrated development environment based on Eclipse, with build, flash, and debug support.
- **STM32Cube firmware packages**: per-family packages containing the HAL and low-level (LL) drivers, middleware (FreeRTOS, USB, FatFS, LwIP, mbedTLS), board support packages (BSP), and example projects.
- **STM32CubeProgrammer**: a flashing and option-byte configuration utility.
- **STM32CubeMonitor**: a real-time variable monitoring tool.

It is this ecosystem — not merely the silicon — that we benchmark against competitors. When we compare "ST versus GD32," we compare the full software stack each vendor provides for a given use case, because that is what an engineer evaluates when choosing a platform.

1.3.2 Firmware memory footprint

The memory footprint of firmware is the amount of non-volatile (flash) and volatile (RAM) memory it occupies once compiled and linked. Flash stores read-only code and constant data; RAM stores read-write data (global and static variables, plus stack and heap at run time).

Footprint is a first-class metric in MCU benchmarking for several reasons. Flash and RAM are fixed, on-chip resources; every kilobyte consumed by the vendor's software stack is a kilobyte unavailable for application logic. A smaller footprint enables the use of a cheaper device with less memory, directly affecting bill-of-materials cost. It also leaves headroom for future features and reduces power consumption in flash-retention scenarios.

Comparing footprint across vendors is non-trivial because each vendor structures, names, and layers its code differently. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, `GPIO_PinInit` on NXP, and `R_IOPORT_Open` on Renesas. A fair comparison must map these correspondences and attribute costs to comparable architectural layers (application, middleware, low-level driver, system runtime).

1.3.3 Linker maps

The linker map is a text file produced by the linker at the end of the build process. For the IAR Embedded Workbench for ARM (EWARM) toolchain [15], this file lists every module and function that was linked into the final image, together with its code size (read-only), constant-data size (read-only), and variable-data size (read-write). It also records the memory sections, placement addresses, and total resource usage.

The linker map is the single input to our footprint analysis tool. By parsing it, we can reconstruct the complete static memory budget of a firmware build without instrumenting or executing the code. This makes the measurement perfectly reproducible: the same source code, compiler version, and optimisation level always produce the same map.

1.4 Work methodology

We adopted a "divide and conquer" approach [7], decomposing the project into well-scoped tasks with clear deliverables and deadlines:

1. **Task 1 — Literature and ecosystem study**: research the STM32Cube ecosystem, competing vendor ecosystems (GD32, NXP, TI, Renesas), linker-map formats, and existing benchmarking practices.

2. **Task 2 — Benchmark scenario design**: define the software stacks to benchmark, select representative boards for ST and each competitor, and design fair, reproducible scenarios with identical observable behaviour on both sides.

3. **Task 3 — Benchmark execution and porting**: build and validate each scenario on the STM32 reference, port it to the competitor platform preserving semantic equivalence, compile both under IAR EWARM, and extract results.

4. **Task 4 — Result comparison and interpretation**: compare per-layer footprint, execution metrics, and qualitative observations; identify where ST excels and where gaps exist; formulate enhancement proposals.

5. **Task 5 — Supportive tool development** (parallel): design and implement MCU Workflow Studio (footprint analysis) and the MCU Benchmark Copilot Agent (benchmark creation and porting assistant) to make the methodology scalable and reproducible.

6. **Task 6 — Documentation and report**: document the methodology, tools, and results in this report.

Conclusion

In this chapter, we introduced STMicroelectronics and its Tunis site, presented the competitive context that motivates cross-vendor benchmarking, and defined the core mission: design benchmark scenarios, execute them on ST and competitor platforms, compare and interpret the results for the marketing team, and propose targeted enhancements where ST can improve. Two supportive tools developed in parallel make this methodology reproducible and scalable. The theoretical foundations — the STM32Cube ecosystem, firmware footprint, and linker maps — provide the vocabulary for the chapters that follow. Chapter 2 and Chapter 3 present the supportive tools, while subsequent chapters present the benchmark results themselves.

Chapter 2

A Cross-Vendor Firmware Footprint Analysis Tool

Introduction

Comparing STMicroelectronics against a competing vendor is, at bottom, a measurement problem: the same workload is built for both platforms and the resulting binaries are weighed against one another. One of the most revealing measurements is the *memory footprint* of the firmware — how much flash and RAM a given software stack consumes once compiled and linked. Footprint is easy to define but tedious to compare fairly across ecosystems, because two vendors rarely name, split, or organise their code the same way. To make this comparison reproducible rather than artisanal, we designed and built a dedicated desktop and command-line application, *MCU Workflow Studio*, that ingests the linker output of two builds and produces a layer-aware, explainable footprint comparison between an STM32 reference and a competitor.

This chapter documents that tool as an engineering contribution in its own right. We first state the problem it solves and the requirements we set for it, then describe its architecture and its end-to-end pipeline, the implementation choices behind its deterministic core and its optional language-model assistance, how it plugs into the wider benchmarking effort, and finally its concrete value and its current limitations.

2.1 The cross-vendor footprint problem

The flash and RAM budget of a microcontroller unit (MCU) is a hard constraint. Every kilobyte of code competes with application logic for a fixed amount of on-chip memory, and it ripples into device cost, power, and the headroom left for future features. When we benchmark a software stack — a Universal Serial Bus (USB) device stack, a real-time operating system integration, and so on —

the footprint each vendor charges for the same functionality is therefore a first-class metric.

Measuring it fairly is the difficult part. A modern build links application code against a vendor software development kit (SDK) whose drivers, middleware, and startup code are structured to that vendor's own conventions. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, as `GPIO_PinInit` on one competitor, and as `fsl_gpio` routines on another; equivalent middleware sits in different modules, and the boundary between "driver" and "system runtime" is drawn differently in each toolchain. Doing the comparison by hand means reading two linker maps side by side, guessing which symbol on one side corresponds to which on the other, and summing kilobytes into comparable buckets — slow, subjective, and impossible to reproduce exactly two weeks later.

We set the tool a small number of firm requirements. It had to be **deterministic and reproducible**, so that the same inputs always yield the same verdict. It had to be **layer-aware**, grouping symbols into architectural layers so that "ST spends more in the hardware abstraction layer (HAL)" can be stated precisely. Its symbol matching had to be **explainable**, attaching a confidence and a reason to every correspondence rather than hiding behind a black box. It had to treat **ST as the reference** against which each competitor is measured. And it had to **export** its results in formats the rest of the report can consume directly.

2.2 Design and architecture

The tool is organised as a thin set of user interfaces wrapped around a deterministic comparison core, with an SQLite-backed knowledge base alongside and an optional, off-by-default block of language-model assistance. Figure 2.1 shows the arrangement. Data flows top to bottom: a build's linker map enters through the ingestion layer, is classified into architectural layers, is compared symbol by symbol in the core, and leaves as a set of exported artefacts.

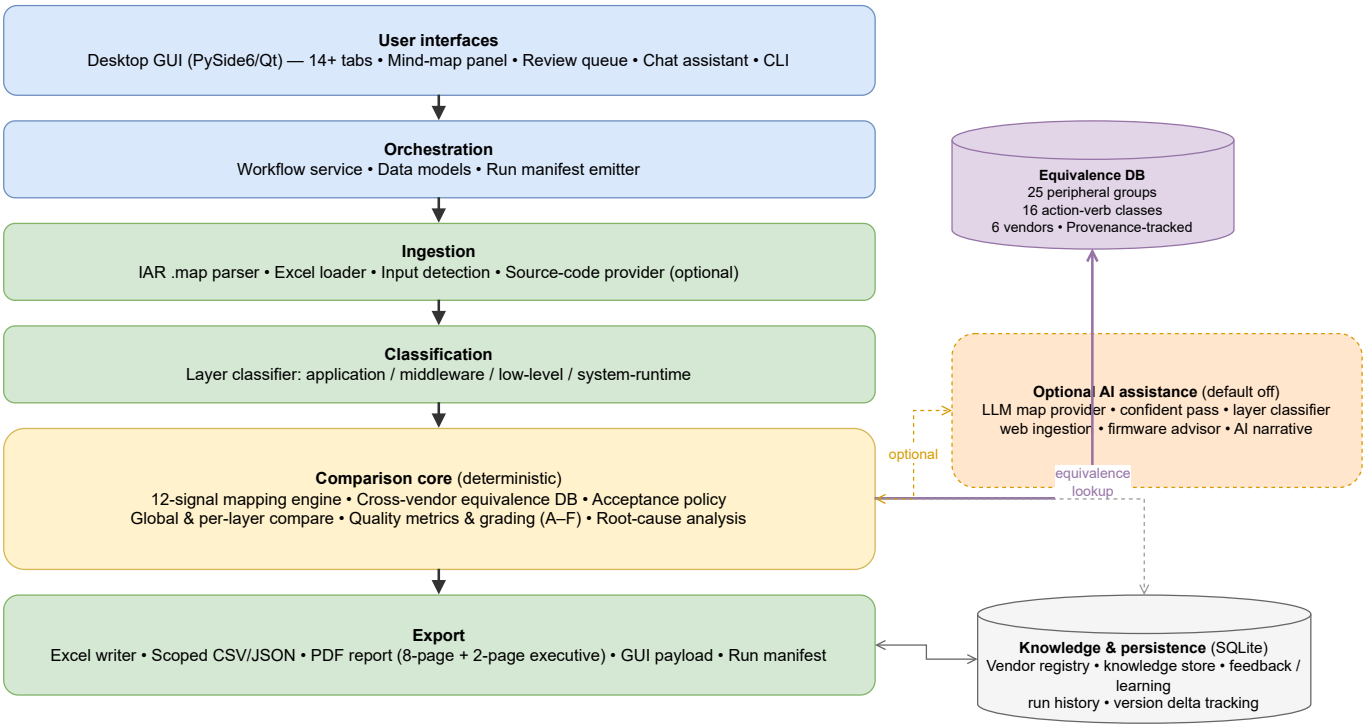

Figure 2.1: Layered architecture of the footprint-analysis tool. A deterministic comparison core (ingestion, classification, mapping, comparison, export) is wrapped by the user interfaces and backed by an SQLite knowledge base; an optional, off-by-default AI-assistance column refines the mapping without ever sitting on the critical path.

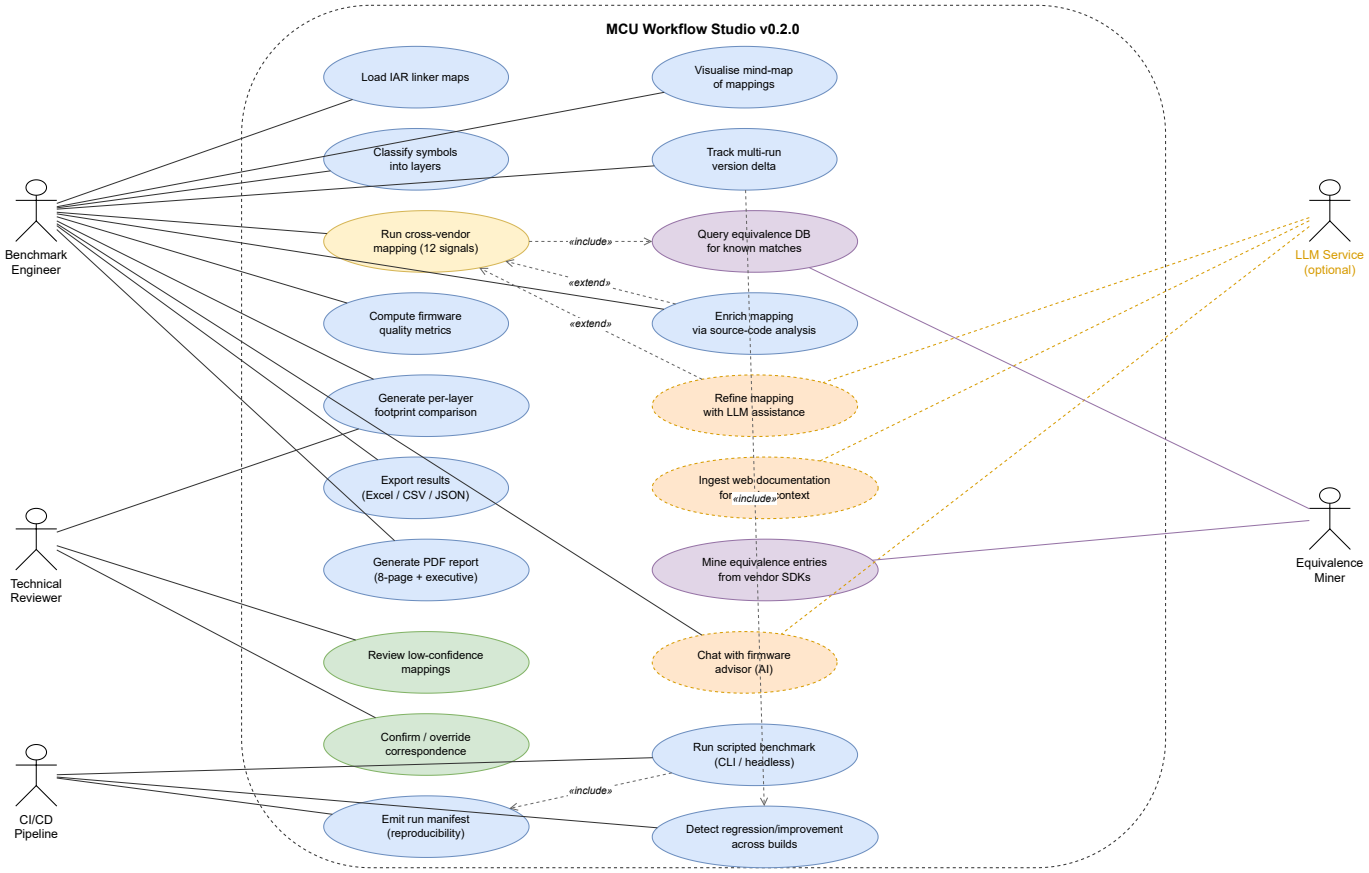

Figure 2.2: UML use-case diagram of MCU Workflow Studio v0.2.0. Five actors interact with twenty use cases grouped by concern: the benchmark engineer drives the core analysis, the technical reviewer validates low-confidence mappings, the CI/CD pipeline runs scripted headless benchmarks, the LLM service (optional, dashed) enriches the mapping, and the equivalence miner maintains the cross-vendor knowledge base.

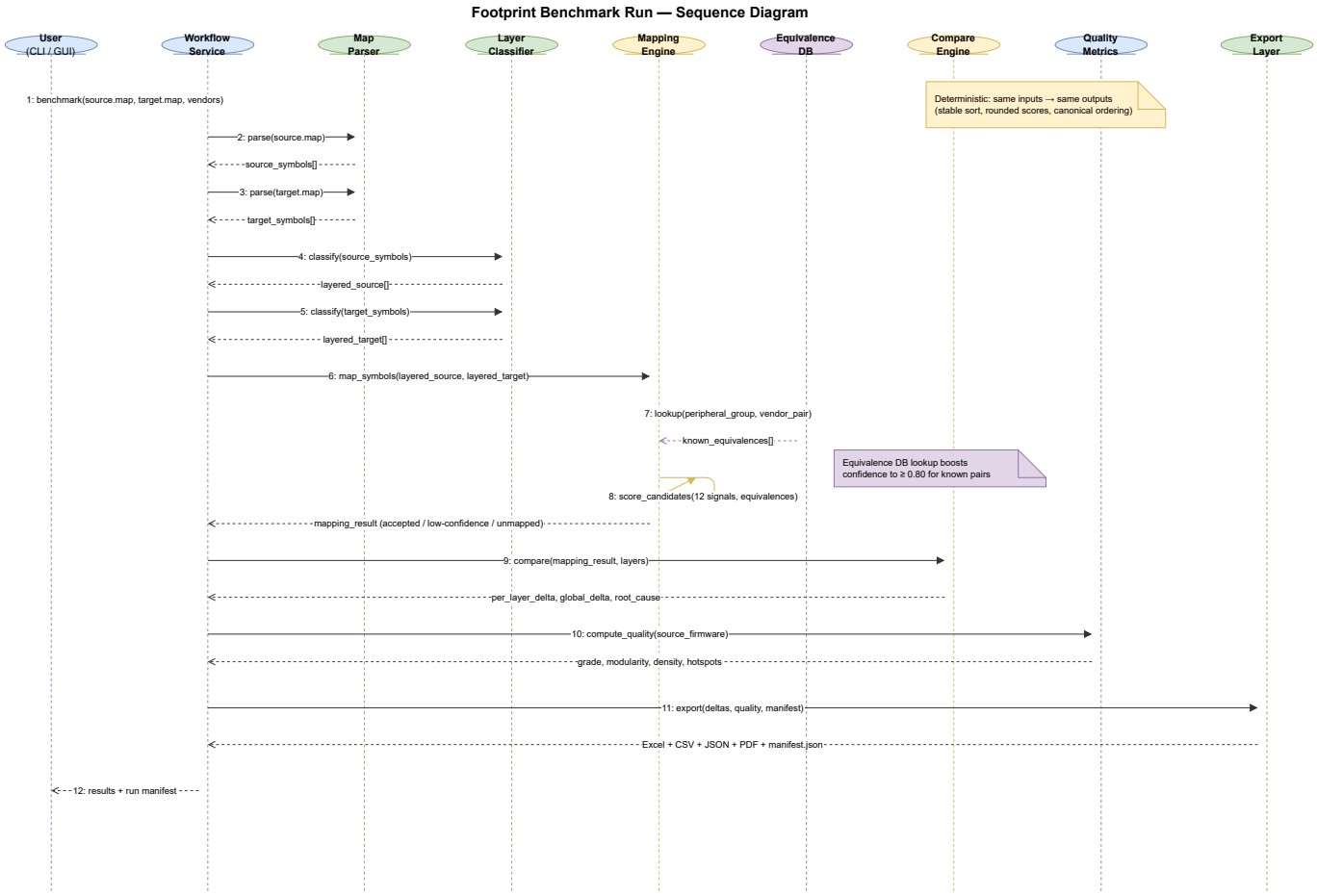

Figure 2.3: UML sequence diagram of a complete footprint benchmark run. The workflow service orchestrates twelve steps: parsing both linker maps, classifying symbols into layers, querying the equivalence database, scoring candidates with twelve weighted signals, computing per-layer deltas, evaluating firmware quality metrics, and emitting the run manifest alongside all export artefacts.

Two interfaces sit on top of the same engine: a desktop graphical user interface (GUI) built with PySide6 (the Qt binding for Python) [9], which provides over fourteen analysis tabs — including interactive charts, a mapping mind-map visualisation panel, a review queue, and a project-scoped chat assistant — and a command-line interface for scripted and reproducible runs. Both call into a single *workflow service* that owns the data models and drives every stage. Persistence is handled by SQLite [13]: a vendor registry of aliases and naming prefixes, a knowledge store of confirmed correspondences, an optional feedback store, and a history of past runs. The block of language-model features on the right is genuinely optional — the deterministic path produces a complete result without it — and we return to it in Section 2.4.3. Table 2.1 lists what the tool consumes and what it produces.

Table 2.1: Principal inputs and outputs of the tool.

Inputs	**Description**
IAR EWARM `.map` (×2)	Linker maps of the source (STM32) and target (competitor) builds; the only required inputs.
Prior workbook (`.xlsx`)	Optional earlier results re-used as a baseline.
Vendor identifiers	Source and target vendor names (default `stm32`/`nxp`) that select the alias and prefix tables.
Outputs	**Description**
`benchmark_results.xlsx`	Per-layer and global flash/RAM comparison with a verdict.
Scoped CSV/JSON/XLSX	Filtered exports for downstream processing.
Review queue (JSON/CSV)	Low-confidence mappings prioritised for manual confirmation.
Mapping diagnostics (JSON)	Per-pair signal breakdown and acceptance decisions.
GUI payload (JSON)	Pre-rendered data for the desktop interface.
PDF report (optional)	Eight-page narrative summary with charts, quality grade, and enhancement roadmap.
Run manifest (JSON)	Tool version, input checksums, equivalence DB hash, and deterministic mode for reproducibility.

2.3 The benchmarking pipeline

A benchmark run is a fixed sequence of stages, sketched in Figure 2.4. The solid path is purely deterministic; the dashed stages are the optional language-model steps, which are disabled by default and can be switched on without changing anything else. After validating its inputs, the tool parses both linker maps into a common in-memory model — build metadata, an overall footprint, and the list of modules and functions with their code and data sizes. Each symbol is then classified into an architectural layer, and the vendor knowledge (aliases, prefixes, prior matches) is prepared.

The heart of the run is the mapping stage, where each STM32 symbol is matched to its closest competitor counterpart by combining several similarity signals into a single, explainable score. The accepted matches are then aggregated: the tool computes the flash and RAM totals per layer and globally for both sides, compares them, and emits a verdict together with recommendations and a root-cause breakdown of where the difference comes from. Finally it persists the quality of the pair and composes its exports; the optional advisor, PDF report, and run-history entry are produced only if requested.

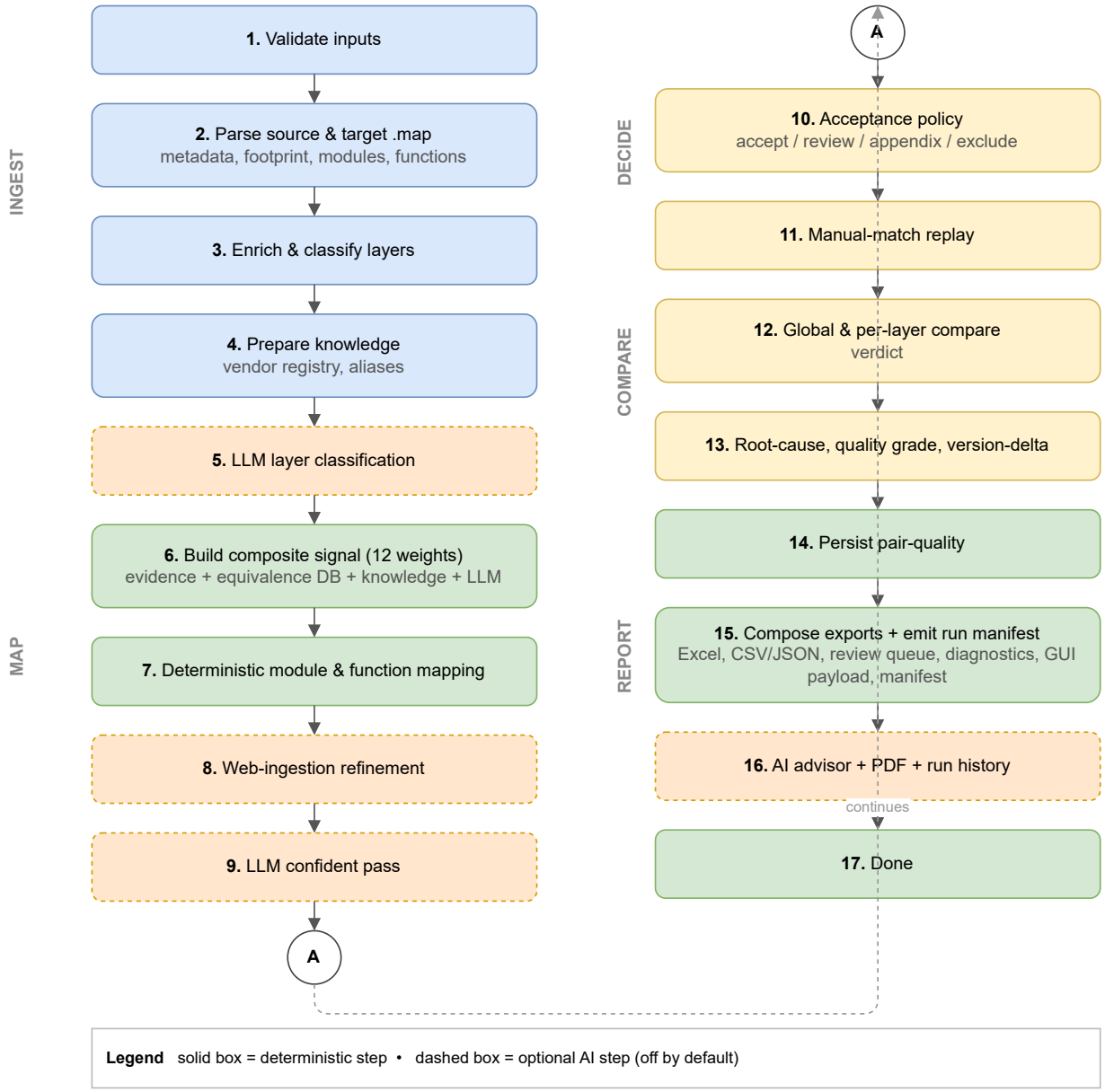

Figure 2.4: End-to-end benchmark pipeline, from linker-map ingestion to report generation. Dashed stages are optional AI-assisted steps that are disabled by default; the solid deterministic path runs end to end on its own.

A second, lighter mode produces a single-map footprint summary — validate, parse, classify, and write a footprint workbook — which is useful when only one build is available or to sanity-check a map before a full comparison.

2.4 Implementation

The tool is written entirely in Python ($\geq$ 3.11) [8], with a small, deliberately conventional dependency set summarised in Table 2.2. We favoured the standard library wherever it was sufficient: all persistence uses the built-in SQLite engine [13], and every network call to a language model goes through the standard `urllib` module rather than a vendor SDK, which keeps the executable small and the dependency surface auditable. The whole application can be packaged into a stand-alone

Windows executable with PyInstaller [14].

Table 2.2: Implementation stack and the role of each external dependency.

Concern	**Technology**	**Role in the tool**
Language	Python $\geq$ 3.11 [8]	Entire code base (version 0.2.0).
Desktop GUI	PySide6 / Qt [9]	14+ tab windows, mind-map panel, chat assistant.
Tabular export	pandas [10]	Scoped CSV/JSON/Excel export.
Excel I/O	openpyxl [11]	Reads prior workbooks, writes results.
Charts & PDF	matplotlib [12]	Figures in the 8-page PDF report.
Persistence	SQLite [13]	Knowledge base, equivalence store, run history.
Packaging	PyInstaller [14]	Stand-alone Windows executable.

2.4.1 Parsing linker maps and classifying layers

The single required input is the linker map emitted by the IAR Embedded Workbench for ARM (EWARM) toolchain [15]. The parser reads this map and reconstructs, for each module and function, the read-only code and data it contributes and the read-write data it reserves. From these we derive the two footprint figures the benchmark cares about: flash occupancy as the sum of read-only code and read-only data, and RAM occupancy as the read-write data. This is a *static* measurement taken from the linked image; it captures what the firmware costs to store, not its dynamic stack or heap behaviour at run time, and we are careful to present it as such.

Each symbol is then assigned to one of four architectural layers — application, middleware, low-level driver, or system runtime — by a deterministic classifier driven by module names, known vendor prefixes, and section attributes. This layering is what lets the final report say not merely that one firmware is larger, but *where* it is larger, which is far more useful when the goal is to understand a vendor's engineering trade-offs. Figure 2.5 details the decision flow.

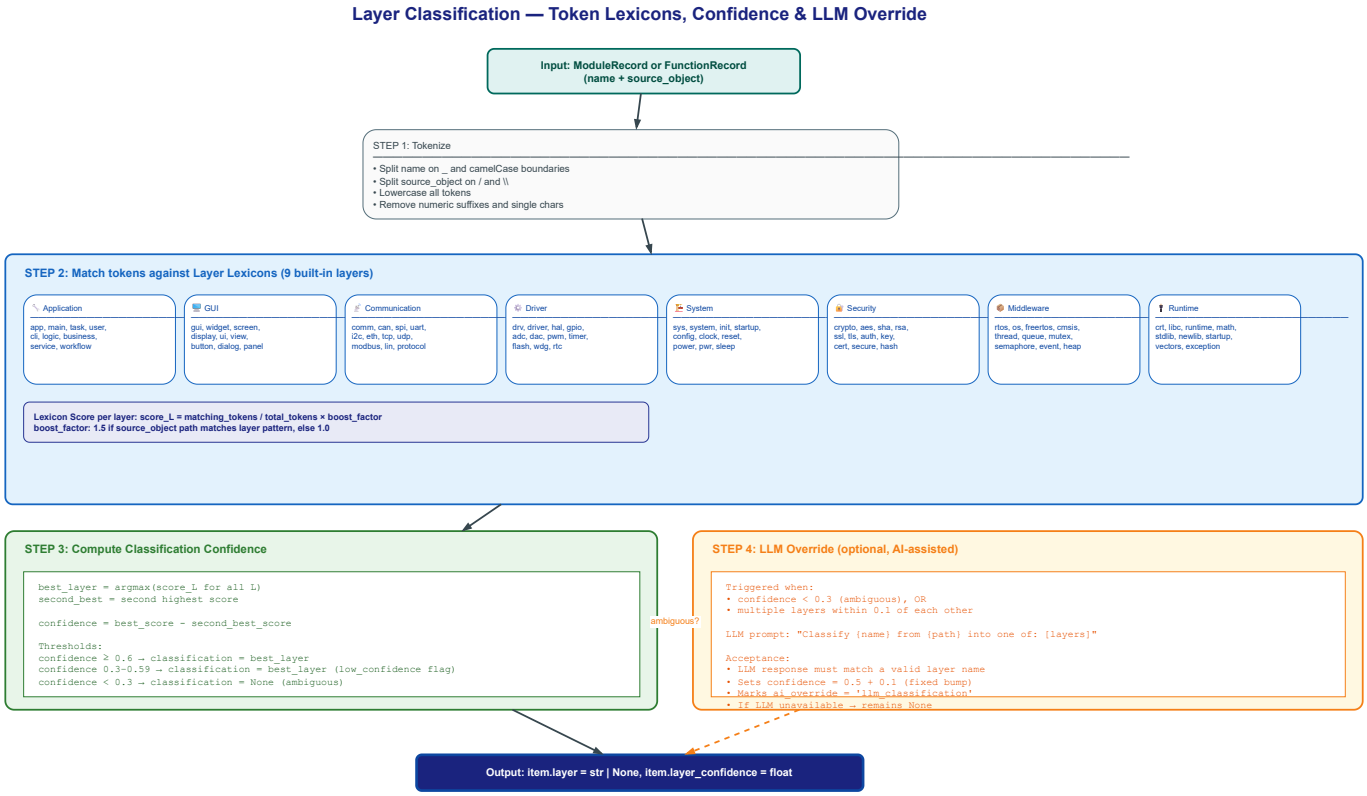

Figure 2.5: Layer-classification decision flow. Each symbol's tokens are matched against priority-ordered lexicons (system, middleware, low-level, application); the first match determines the layer with an associated confidence.

2.4.2 The multi-signal mapping engine

Matching ST symbols to their competitor counterparts is the part that, done manually, costs the most time and is the least repeatable, so it received the most care. Rather than rely on name similarity alone — which fails precisely when two vendors name the same function differently — the engine scores each candidate pair by combining twelve weighted signals into a single, explainable confidence value. Role similarity and alias similarity carry the most weight (0.18 each), followed by name similarity and operation similarity (0.12 each), external references and signature cues (0.10 each), footprint proximity (0.08), object similarity (0.06), context cues (0.04), and three knowledge-base priors (history, board-pair, scenario — contributing the remaining 0.02). The weights sum to exactly 1.0. Figure 2.6 shows the relative weights, and Figure 2.7 expands the computation and penalty mechanics of each signal.

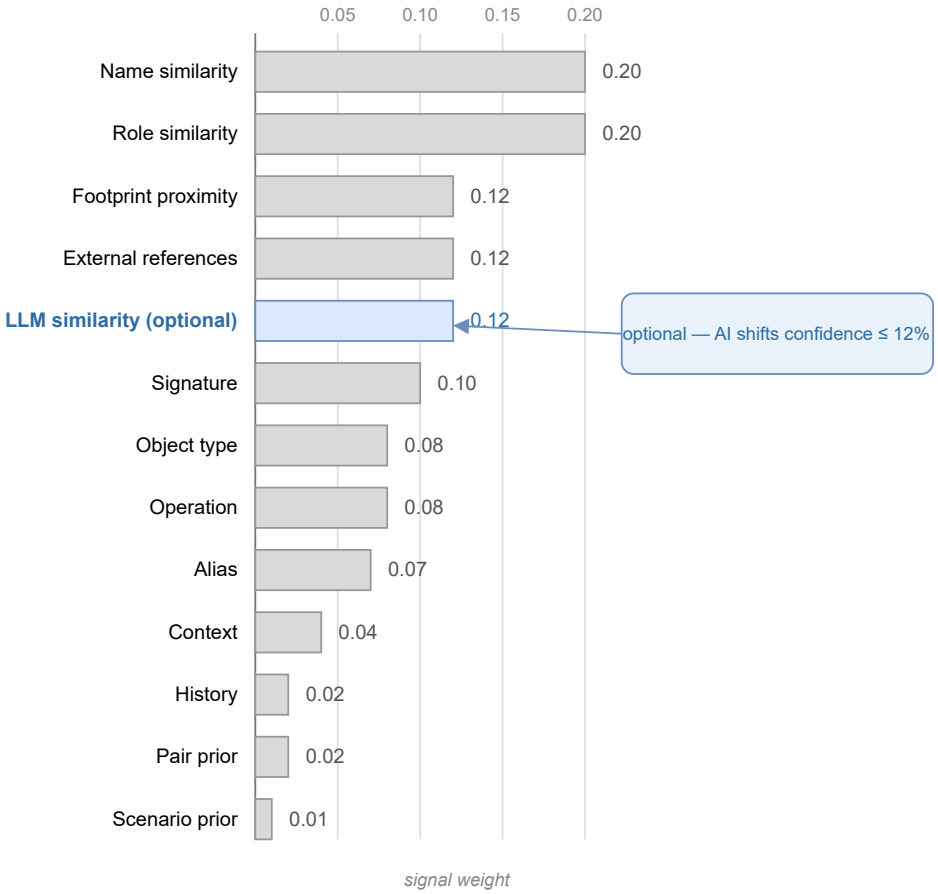

Figure 2.6: Relative weights of the signals combined by the deterministic mapping scorer. Name and role similarity dominate; the optional LLM-similarity signal (highlighted) is capped so that AI assistance can shift a candidate's confidence by at most twelve percent.

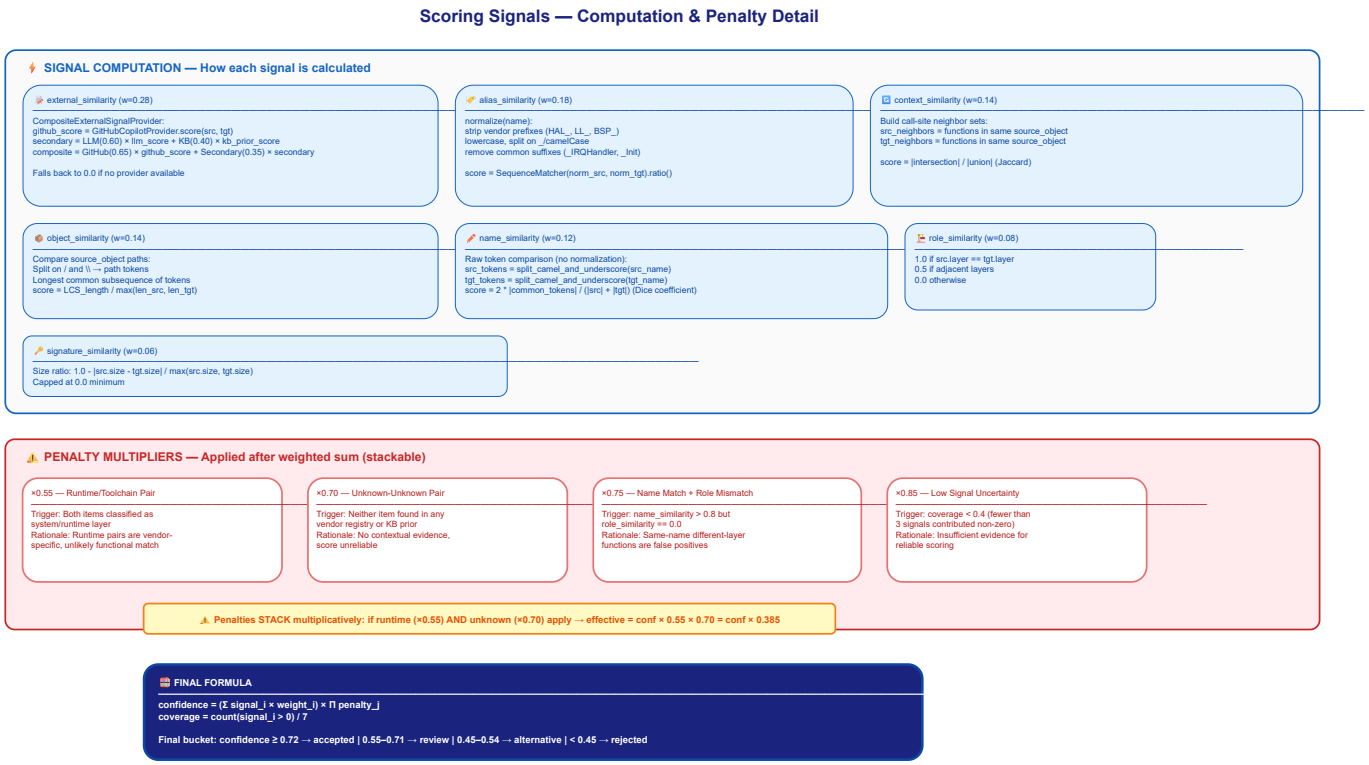

Figure 2.7: Detailed signal computation and penalty system. Seven evidence signals are computed for each candidate pair; penalties for size mismatch and layer conflict reduce the composite score before the acceptance threshold is applied.

The resulting confidence drives a transparent acceptance policy. A correspondence is accepted automatically once its confidence reaches 0.72, queued for manual review above 0.55, kept as a weaker alternative above 0.45, and an unmatched symbol is retained on its own only above 0.80. The engine also records the cardinality of each match — one-to-one, one-to-many, or many-to-one — which matters because vendors routinely split or merge functionality differently, and a one-to-many match is itself a finding about how the two stacks are structured. Every decision, with its per-signal breakdown, is written to the mapping diagnostics so that a reviewer can see exactly why two symbols were paired. Figure 2.8 illustrates the complete flow from candidate generation through pruning, scoring, and threshold-based bucketing.

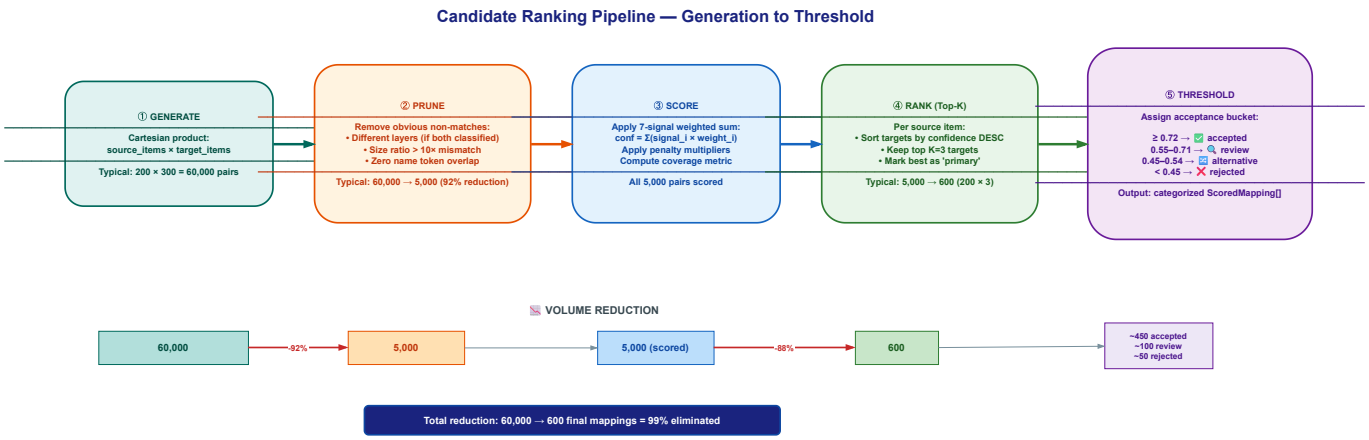

Figure 2.8: Candidate generation, ranking, and threshold flow. The Cartesian product of source and target items is pruned by fast rejection filters, scored on all signals, and bucketed into accepted, review, alternative, and unmatched categories by confidence thresholds.

2.4.3 Optional AI assistance

The tool is best described as a deterministic core with *optional* language-model assistance, not as an "AI tool" in the sense of one that cannot work without a model. Every language-model feature is opt-in and disabled by default, and the engine produces a complete, reproducible result with all of them switched off. When they are enabled, they act in three bounded ways. First, a large language model (LLM) can supply one additional similarity signal during mapping; as Figure 2.6 makes explicit, that signal is weighted and capped so it can move a candidate's confidence by at most twelve percent, and the run stays within a fixed call budget. Second, a "confident pass" may confirm or reject borderline matches, and an advisor may suggest where STM32 firmware could be tightened. Third, the desktop interface offers a chat assistant for exploring a comparison in natural language.

These features can be served by several interchangeable back-ends, listed in Table 2.3, selected automatically in a fixed order or pinned explicitly. They range from a hosted GitHub Copilot endpoint [16] to a fully offline Ollama server [19], so the assistance can run without any data leaving the machine. If a back-end is unavailable or returns something unusable, the tool degrades gracefully to its deterministic output rather than failing. Figure 2.9 shows the provider abstraction and the auto-detection fallback chain.

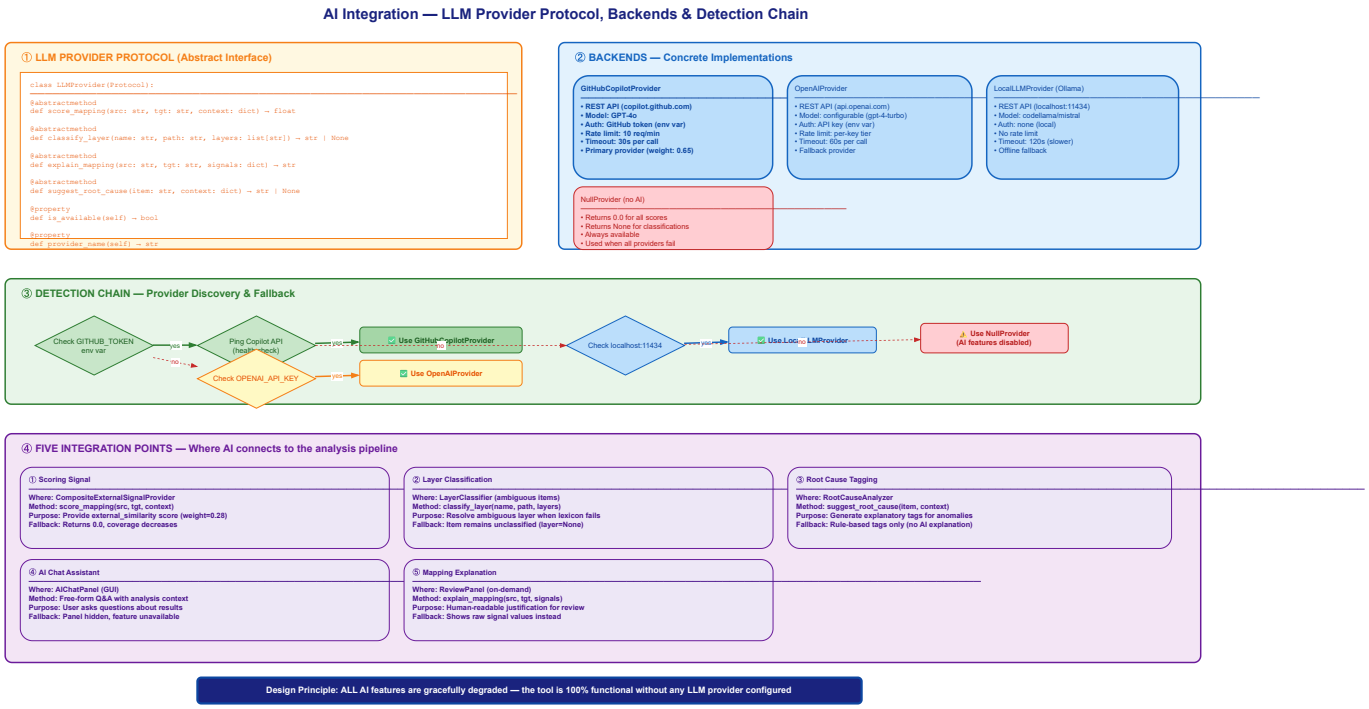

Figure 2.9: AI integration flow. A common protocol abstracts all back-ends; an auto-detection chain probes them in priority order (Copilot, OpenAI, Ollama) and falls back gracefully to purely deterministic operation if none is available.

Table 2.3: Optional language-model back-ends. All are disabled by default; the tool runs fully without them.

Back-end	Default model	Selection / requirement
GitHub Copilot [16]	Claude (Anthropic) [17]	OAuth device flow; first in the auto-detect order.
OpenAI [18]	GPT-4o	`OPENAI_API_KEY` environment variable.
Ollama [19]	Llama 3.1 (8B)	Local server; fully offline.
GitHub Models [20]	gpt-4.1-mini	Default for the narrative assistant.

2.4.4 Persistence and learning

All state lives in local SQLite databases: the vendor registry, the store of confirmed correspondences, a cache of assistant responses, and a history of runs. A manual confirmation made during review can be replayed on later runs, so the tool improves as it is used on a given vendor pair. We treat this learning, together with the web-ingestion and run-history features, as useful but experimental, and none of it is on the critical path of a comparison.

2.4.5 Cross-vendor equivalence database

A critical addition to the scoring engine is the *cross-vendor equivalence database*, which encodes deterministic knowledge about which modules and functions serve the same peripheral function

across vendor SDKs. When two symbols are confirmed equivalents by this database, their confidence is boosted to the high-confidence threshold regardless of naming differences.

The database covers twenty-five peripheral groups (GPIO, UART, SPI, I^2C, Timer, Clock, Power, DMA, USB Device, USB Host, Flash, ADC, DAC, CAN, Ethernet, RTC, Watchdog, CRC, RNG, I^2S, Pin Mux, Interrupt, Cortex core, BSP/Board, System Init, Startup, and PWM) with token-pattern entries for six vendors: STM32, NXP, Renesas, TI, GigaDevice, and Infineon. It also encodes sixteen action-verb equivalence classes (init/deinit, start/stop, transmit/receive, set/get, reset, callback, register/ unregister, lock/unlock, suspend/resume), so that `HAL_GPIO_Init` and `GPIO_Setup` are recognised as serving the same role.

The database is designed as an offline, provenance-tracked artefact: an equivalence miner extracts entries from pinned vendor SDK repositories (resolved to a specific commit SHA for reproducibility), records provenance for every entry, and produces a content-hashed mining manifest. This ensures that the equivalence knowledge used in any given run is auditable and frozen — it cannot drift silently between executions.

2.4.6 Source-code signal provider

When both source and target IAR EWARM project directories are available, an optional source-code signal provider enriches the mapping with three additional signals derived from the actual C source files: function signature similarity (parameter count and return-type patterns), call-graph structural similarity (shared callee patterns), and include-graph co-location (functions in files with similar `#include` dependencies). This provider operates as an external-signal plug-in to the scoring engine and is disabled by default; when enabled, it provides a richer correspondence basis for firmware that is available locally, without requiring any network access or language model.

2.4.7 Firmware quality metrics and grading

Beyond comparing two builds, the tool computes firmware quality metrics for the source (STM32) firmware independently: total flash and RAM, module and function counts, average and maximum module size, layer-balance score, modularity score, code density, HAL abstraction ratio, middleware ratio, application ratio, large-module count, dead-weight estimate, optimisation potential, and an overall quality grade (A through F). These metrics appear in the PDF report and help the technical marketing team assess not just how STM32 compares to a competitor, but how well the STM32 firmware itself is structured.

2.4.8 Run manifest and reproducibility

Every benchmark run emits a *run manifest* that captures the tool version, the deterministic mode (strict or assisted), the schema version, input file checksums, and a hash of the equivalence database state at the time of execution. Together with stable sort tie-breaks, rounded scores, and canonical export ordering, this information is intended to make a run traceable and reduce output variation when frozen inputs and configuration are processed again. These mechanisms provide a basis for reproducibility testing; however, byte-for-byte equivalence across environments has not yet been established by a dedicated validation campaign.

2.5 Integration into the benchmark workflow

The tool occupies a precise place in our method. Once a software stack has been built for an STM32 board and for the competing vendor's board under identical scenarios, each build's IAR linker map is fed to the tool as the source and the target. Its per-layer and global footprint figures, its verdict, and its root-cause breakdown are exactly the numbers the per-vendor benchmarking chapters of this report quote when they weigh ST's flash and RAM cost against a competitor's for a given stack. In other words, the footprint comparisons reported later are not assembled by hand: they are produced, with an auditable trail, by the tool described here, which is why we document it before presenting any results that depend on it.

2.6 Usage and outputs

A comparison can be driven either from the desktop interface or, for reproducible and scriptable runs, from the command line. A typical invocation names the two maps and their vendors and points at an output directory, as in Listing 2.1.

```
mcu-workflow-cli benchmark \
  --source stm32usbhidclassproject.map --source-vendor stm32 \
  --target nxpusbhidclassproject.map   --target-vendor nxp \
  --out results/
```

Listing 2.1: Command-line invocation of a footprint benchmark.

The run leaves behind the artefacts of Table 2.1: a results workbook with the per-layer and global comparison, scoped CSV/JSON exports, a prioritised review queue of the matches that warrant a human look, the full mapping diagnostics, and — on request — an eight-page PDF report with an executive summary, source-versus-target charts, an architecture-and-module breakdown, a gap analysis, and recommendations. The samples we used to develop and validate the tool are

themselves a representative comparison: a USB human interface device (HID) class project built for an STM32U575 board against the equivalent project built for an NXP MCXN947 board [21], both compiled with the same IAR toolchain so that only the vendor software stacks differ.

2.7 Engineering value and limitations

MCU Workflow Studio consolidates operations that would otherwise require manual inspection of two linker maps: symbol extraction, architectural classification, candidate mapping, footprint aggregation, and export. Its review queue and per-match diagnostics preserve the points that still require engineering judgement, while the run manifest records the inputs and knowledge-base state used for a comparison. The tool therefore provides a more structured and traceable analysis process, although this report does not quantify the time saved or establish that every mapping is correct.

The twelve-signal scoring engine and the cross-vendor equivalence database allow the tool to propose mappings without requiring language-model assistance. Their presence should not be confused with measured mapping accuracy: a labelled validation corpus covering all supported vendors has not yet been reported. Likewise, firmware quality grades and optimisation-potential estimates are heuristic indicators intended to guide inspection, not independently validated measurements of software quality.

We are equally clear about what the tool does not do. It reads IAR EWARM linker maps; builds produced by other toolchains are out of scope for now, and broadening the parser is the most obvious avenue for future work. Its footprint figures are static, derived from the linked image rather than from execution. The quality grades and optimisation-potential estimates that the report produces are heuristic indications, not measured quantities, and we label them accordingly wherever they appear. Finally, the language-model features, while useful, are optional and still maturing; the trustworthy core of the tool is, and is meant to remain, fully deterministic.

Conclusion

We designed and built MCU Workflow Studio to structure cross-vendor footprint analysis. Its core parses two IAR linker maps, classifies symbols into architectural layers, proposes cross-vendor correspondences, and reports per-layer and global flash and RAM differences with a diagnostic breakdown. The run manifest records the inputs and configuration needed for traceability and later reproducibility testing. Firmware quality metrics and optional language-model assistance support review, but neither replaces validation against labelled data and source artifacts. The chapters that follow use the tool's footprint outputs under these limitations.

Chapter 3

An Autonomous AI Agent for Benchmark Campaigns

Introduction

The previous chapter presented a tool that analyses the *output* of a benchmark — two linker maps produced by completed builds. However, producing those builds in the first place is itself a substantial engineering task. Each new cross-vendor comparison requires creating a benchmark project for the competitor platform, porting the STM32 reference code while preserving its exact runtime semantics, configuring the IAR Embedded Workbench for ARM (EWARM) project correctly, and debugging the inevitable compiler, linker, and runtime issues that arise when targeting unfamiliar silicon. Done manually, this process is slow, error-prone, and difficult to keep consistent across many vendor comparisons.

To address this, we designed and built the *MCU Benchmark Copilot Agent* — a custom GitHub Copilot agent system that operates inside Visual Studio Code (VS Code) and *autonomously* drives end-to-end benchmark campaigns: from scenario resolution and firmware acquisition, through project creation, IAR build automation, evidence-driven debugging, measurement extraction, and semantic-equivalence verification [16, 22]. Unlike a passive assistant, the agent executes builds, fetches missing materials from official vendor sources, applies fixes, and iterates — asking the user only before programming physical hardware. Where MCU Workflow Studio analyses what *exists*, the agent system *creates* what does not yet exist, and it does so under strict rules that enforce semantic equivalence and benchmark fairness at every step. The two tools are complementary: the agent produces validated builds, and the studio analyses their footprint.

This chapter documents the agent as an engineering contribution. We describe the problem it addresses, its layered architecture (agents, skills, automation tools, configuration registry), its deterministic campaign workflow, the always-on policies that govern its behaviour, and its integration

with the broader benchmarking effort.

3.1 The benchmark engineering problem

A single STM32-versus-competitor benchmark involves several non-trivial steps. The STM32 reference project must first be understood in full — task structure, timing, peripheral usage, reporting format, and observable behaviour. A corresponding project must then be created for the target platform, mapping every STM32 hardware abstraction layer (HAL) call to the competitor's equivalent application programming interface (API) without altering the benchmark's semantics. The resulting project must be configured for the IAR toolchain [15] with the correct device selection, startup file, linker configuration, include paths, and compiler defines. Once it compiles, runtime issues — wrong interrupt vectors, misconfigured clocks, broken universal asynchronous receiver-transmitter (UART) output — must be diagnosed from real evidence rather than guesswork. And throughout, the comparison must remain fair: same optimisation level, same measurement scope, same observable output format.

Doing this for one vendor pair and one software stack is manageable. Repeating it for multiple vendors (GigaDevice, NXP, Renesas, Texas Instruments, Infineon, Nordic, Microchip) across multiple stacks (USB device, FreeRTOS, CoreMark) quickly becomes the dominant time cost of the benchmarking campaign. Each port also carries a risk of *semantic drift* — subtle changes in task priorities, timing, or synchronisation that invalidate the comparison without being immediately visible.

We needed a system that could enforce the methodology consistently across every port, refuse to guess when evidence was missing, execute deterministic build and measurement workflows without manual intervention, and make its reasoning transparent. A general-purpose large language model (LLM) cannot do this out of the box: it lacks the domain-specific checklists, the structured routing, the automation tooling, and the policy of refusing to claim success without hardware evidence. A custom agent system, built on VS Code's Copilot extensibility framework, gave us the ability to encode our benchmarking methodology — and to *execute* it autonomously — directly within the development environment [23].

3.2 Architecture

The system is structured as four cooperating layers: specialist *agents* that handle reasoning and delegation, reusable *skills* that encode methodology as checklists, *automation tools* (PowerShell and Python scripts) that perform deterministic operations, and a *configuration registry* (YAML files) that provides ground truth about boards, MCUs, vendors, toolchains, and measurement profiles.

A shared set of always-on rules governs all layers. All configuration lives in a `.github/` directory within the benchmark workspace — no compiled code, no external server, no deployment step. The agents run inside VS Code's GitHub Copilot Chat extension [16] and have access to the workspace file system, terminal, web, and search capabilities.

Figure 3.1 illustrates the layered arrangement. A user request enters the coordinator agent, which resolves the scenario, classifies the task, acquires missing materials, and delegates to the appropriate specialist agent or invokes a skill workflow. Specialist agents operate in isolated context, use the automation tools for deterministic operations, and return structured results.

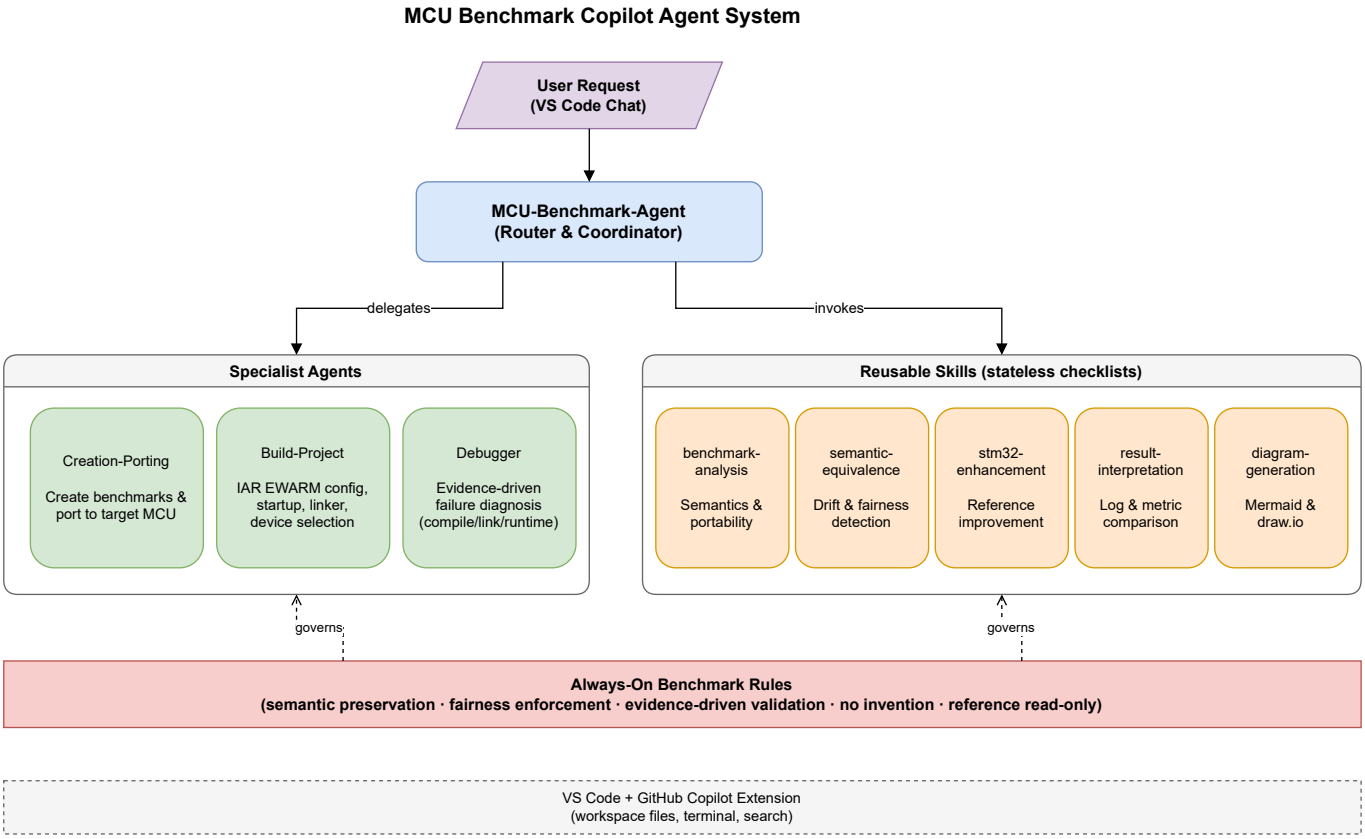

Figure 3.1: Architecture of the MCU Benchmark Copilot Agent system. The coordinator resolves scenarios and delegates to specialist agents; automation tools handle deterministic build/measurement operations; all layers operate under the shared always-on benchmark rules and consult the configuration registry.

3.2.1 The four specialist agents

Each agent has a defined responsibility, a restricted tool set, an autonomy contract, and a structured output contract. Table 3.1 summarises them.

Table 3.1: Specialist agents and their responsibilities.

Agent	**Role**	**Responsibility**
MCU-Benchmark-Agent	Autonomous coordinator	Resolves scenarios from partial input, acquires missing materials, orchestrates end-to-end campaigns, delegates to specialists. Full tool access including web and execute.
Creation-Porting	Project generation	Creates new benchmark projects from scratch or ports existing benchmarks in any direction (STM32→competitor, competitor→STM32, vendor→vendor, board→board). Acquires firmware autonomously.
Build-Project	Build automation	Autonomously executes IAR builds, classifies failures, applies project-level fixes, and rebuilds — without user intervention for non-hardware tasks.
Debugger	Failure diagnosis	Autonomously diagnoses compiler, linker, and runtime failures; extracts measurements from logs and captures; applies minimal fixes and revalidates.

The coordinator's routing decision tree is deterministic: project understanding goes to the *benchmark-analysis* skill; cross-vendor porting goes to *Creation-Porting*; build execution and configuration go to *Build-Project*; failures accompanied by real evidence go to *Debugger*; and result interpretation uses the dedicated skill. This routing prevents a general-purpose model from attempting tasks outside its configured expertise and ensures each specialist receives only the context it needs.

Figure 3.2 presents a complete use case diagram of the system, showing all actors, the twenty internal capabilities, and the hardware approval boundary that separates autonomous operations from those requiring user confirmation.

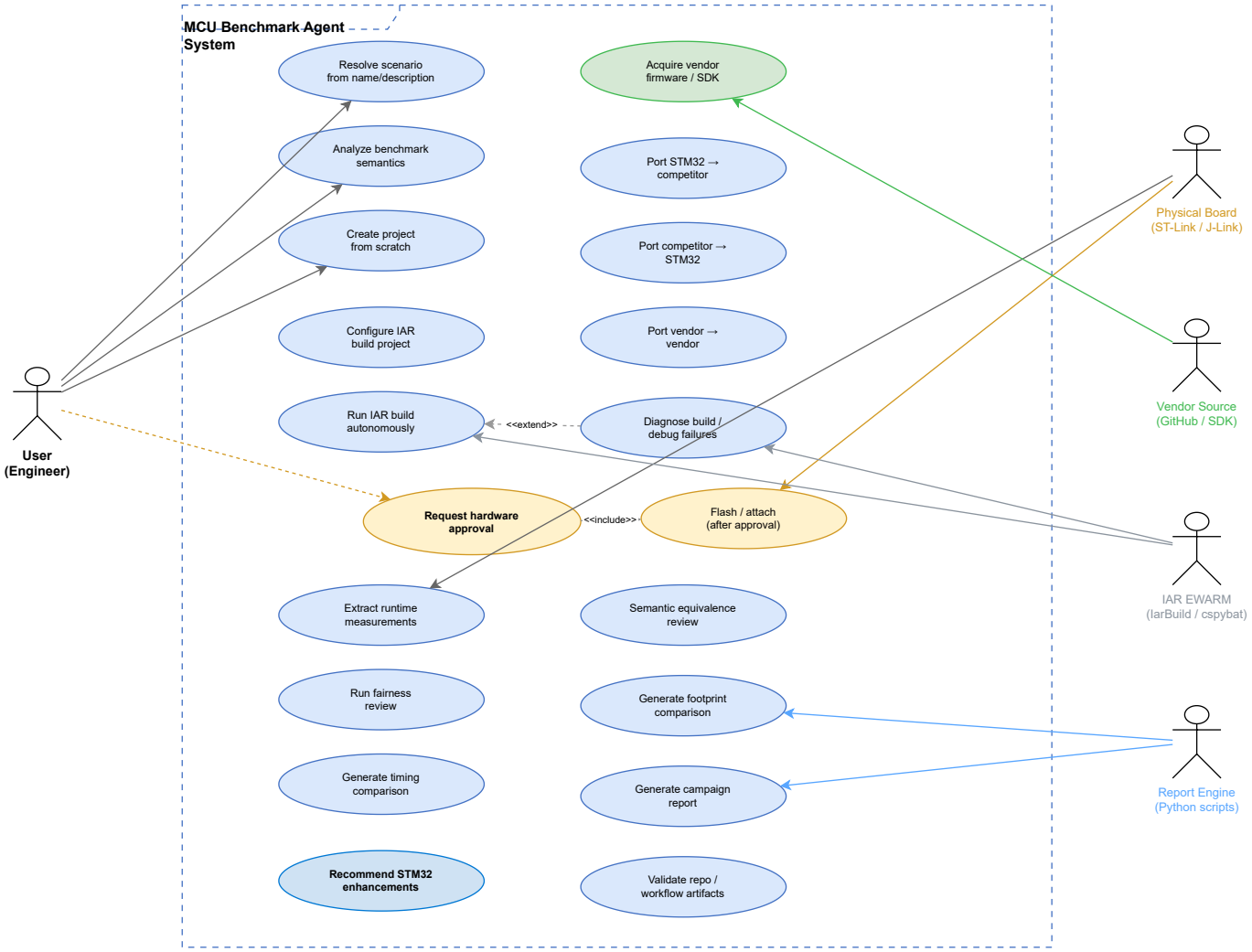

Figure 3.2: UML use case diagram of the MCU Benchmark Copilot Agent system. Blue ellipses represent autonomous use cases; yellow ellipses require user approval; green indicates external acquisition. External actors include the user, physical board, vendor source repositories, IAR EWARM toolchain, and the report engine scripts.

3.2.2 The six reusable skills

Skills encode repeatable workflows as structured Markdown checklists. Unlike agents, they do not receive isolated context or delegated reasoning — they provide structure, not autonomy. Table 3.2 lists them.

Table 3.2: Reusable skills and their purposes.

Skill	Purpose	Primary users
benchmark-analysis	File classification, semantics extraction, and portability split of an existing project.	Coordinator, Creation-Porting
semantic-equivalence	Reference-versus-target behaviour comparison and drift detection.	Coordinator, Creation-Porting
iar-template-resolution	Resolve, acquire, and integrate IAR project templates per board type using a deterministic lookup algorithm.	Creation-Porting, Build-Project
stm32-enhancement	Gap-driven recommendations for improving STM32 reference portability.	Coordinator
result-interpretation	Comparative interpretation of benchmark logs, measurements, and result files.	Coordinator, Debugger
diagram-generation	Mermaid and draw.io workflow, architecture, and presentation diagrams.	Coordinator

3.2.3 Automation tools

A critical addition to the system is a library of twelve deterministic scripts (`.github/tools/`) that the agents invoke via the terminal for operations that must produce repeatable results. Table 3.3 lists the key scripts.

Table 3.3: Automation tools in the agent system.

Script	Language	Purpose
`run_iar_build.ps1`	PowerShell	Wraps IarBuild.exe with structured output, exit-code handling, and log capture.
`discover_iar.ps1`	PowerShell	Deterministic IAR installation discovery at known paths.
`acquire_iar_template.ps1`	PowerShell	Downloads and materialises IAR board templates from official vendor sources.
`run_cspy_capture.ps1`	PowerShell	Runs cspybat.exe batch debug sessions and captures measurement output [25].
`collect_measurements.py`	Python	Parses UART, SWO, and C-SPY output into structured measurement YAML.
`normalize_measurements.py`	Python	Normalises raw measurements for cross-platform comparison.
`compare_measurements.py`	Python	Computes deltas and fairness-aware comparisons between platforms.
`normalize_build_log.py`	Python	Structures IAR build diagnostics into classified error/warning lists.
`parse_map_file.py`	Python	Extracts memory-usage data from IAR linker map files.
`generate_benchmark_report.py`	Python	Produces structured campaign reports from collected artifacts.
`validate_benchmark_repo.py`	Python	Validates workspace structure and campaign artifact completeness.
`validate_iar_templates.py`	Python	Checks materialized IAR templates against expected file inventories.

These scripts ensure that build commands, measurement parsing, and comparison logic execute identically regardless of which agent invokes them or when. The agents handle reasoning and decision-making; the scripts handle deterministic execution.

3.2.4 Configuration registry

The YAML-based configuration registry (`.github/config/`) provides ground truth that the agents consult rather than hallucinate. Table 3.4 lists its files.

Table 3.4: Configuration registry files.

File	Content
`boards.yaml`	Board registry with official identifiers, MCU, core, probe, measurement capabilities, and aliases for all supported evaluation boards.
`mcus.yaml`	MCU specifications: flash, RAM, clock, peripherals, startup and linker file references.
`vendors.yaml`	Vendor metadata: supported families, HAL framework, RTOS defaults, wrapper model, and board naming conventions for ST, TI, NXP, Renesas, GigaDevice, Infineon, Nordic, and Microchip.
`toolchains.yaml`	IAR EWARM binary paths, discovery order, build-command patterns, and configuration areas.
`iar_templates.yaml`	IAR project template registry with acquisition status, source URLs, and resolution algorithm metadata.
`vendor_sources.yaml`	Official firmware acquisition URLs and SDK references per vendor.
`measurement_profiles.yaml`	Standard measurement definitions (task-switch cycles, IRQ latency, flash/RAM footprint) with fairness risks and required evidence.
`reporting.yaml`	Output format templates for campaign reports.

The registry solves a fundamental problem with LLM-based agents: when the model does not know a board's pin mapping or a vendor's SDK repository URL, it might invent one. By encoding verified facts in YAML and instructing the agents to consult these files, we eliminate an entire class of hallucination errors.

3.3 Supported workflow directions

A key design decision was to support benchmarking in *all* directions, not only the obvious STM32-to-competitor port. The system handles six workflow directions:

1. **STM32 → competitor** — port an STM32 reference benchmark to a competitor target (the primary use case).

2. **Competitor → STM32** — port a competitor's reference project to an STM32 target (useful when the competitor has a ready example that ST lacks).

3. **Vendor A → Vendor B** — port between any two non-ST vendors.

4. **Board A → Board B** — replicate a ready scenario on a different board (same or different vendor).

5. **Scenario description → new project from scratch** — create a complete benchmark project from only a textual scenario description, with no pre-existing reference code.

6. **Firmware/benchmark name only → acquire and build** — the user provides only a firmware package name (e.g. `USB_Device_HID`); the agent resolves the scenario, acquires the firmware from official sources, and proceeds autonomously.

In all directions, the same semantic preservation and fairness rules apply. The agent generates a new target project rather than modifying any reference, and it preserves benchmark semantics (observable behaviours), not vendor API names.

3.4 Autonomy model and approval gates

The agent operates autonomously by default for all workspace inspection, file generation, build execution, debug preparation, material acquisition, and measurement extraction. Table 3.5 clarifies the boundary.

Table 3.5: Autonomy model: autonomous actions versus approval-required actions.

Autonomous (no confirmation)	**Requires user approval**
Read, search, and inspect any workspace file	Flash or program a physical board
Create, edit, or reorganise application-layer project files	Erase or reset a physical device
Run IarBuild.exe and other local build commands	Attach a live debug probe to real hardware
Parse build logs, linker maps, UART output, and debugger captures	Modify vendor baseline file contents (startup, linker, HAL, BSP, middleware)
Fetch firmware, CMSIS headers, BSP, and documentation from official vendor sources	—
Apply minimal project-configuration fixes and rebuild	—
Run cspybat.exe in simulation/batch mode	—

After the user grants one explicit board-programming approval in a session, the agent continues autonomously through the remaining flash → debug → measure → report sequence for that board without re-asking. This design maximises throughput while maintaining a hard safety boundary at the point where actions become physically irreversible.

3.5 Board firmware baseline immutability

A distinctive policy of the system is the *Board Firmware Baseline Immutability Rule*. Vendor SDK, HAL, Low-Layer (LL), Board Support Package (BSP), startup, linker, middleware, and driver files constitute a read-only baseline infrastructure. The agents integrate benchmarks exclusively at the application layer, preserving the board firmware baseline intact. Table 3.6 classifies the writable and protected surfaces.

Table 3.6: Surface classification under the baseline immutability rule.

Surface	**Policy**	**Examples**
Application layer	Writable (default)	`main.c`, `benchmark_tasks.c`, app wrappers and adapters
Middleware / RTOS	Protected (read-only)	FreeRTOS kernel, USB stack, filesystem
HAL / BSP / Startup / Linker	Protected (read-only)	`startup_*.s`, `*.icf`, `stm32xx_hal_*.c`
Build surface	Configure only	`.ewp` references, include paths, defines, device selection

The key distinction is between *selection* and *modification*. The agent may freely *select* which existing vendor file to reference in the project (e.g. choosing `startup_stm32u575xx.s` from the SDK). However, *editing the content* of that file — to add instrumentation, change the vector table, or alter initialisation sequences — is an approval-required deviation. If a benchmark cannot be implemented through application-layer integration alone, the agent reports the need and requests explicit approval before proceeding. Any approved deviation is recorded in the campaign's `fairness_baseline.yaml` for audit.

This rule ensures that benchmark results reflect the vendor's actual firmware infrastructure, not a customised version that would invalidate the comparison.

3.6 Always-on benchmark rules

All agents and skills inherit a shared policy layer — the *always-on benchmark rules* — that constrains the system's behaviour regardless of which specialist is active. These rules encode the core principles of our benchmarking methodology:

- **Reference is read-only.** The STM32 reference project is never modified unless the user explicitly requests it; new target projects are generated instead.
- **Semantic preservation.** A seventeen-point checklist governs what must be identical between reference and target: task count and creation order, priorities, delays, periodicity, synchronisation primitives, suspend/resume and yield behaviour, counters, state transitions, measurement scopes, reporting format, and error behaviour.
- **No invention.** The system does not invent vendor APIs, pin mappings, peripheral choices, or board details. Uncertain hardware details are marked as `TODO` rather than guessed. The configuration registry is consulted for verified facts.
- **Evidence-driven validation.** No claim of runtime success is made without actual compiler output, runtime logs, UART captures, or debugger observations.
- **Fairness enforcement.** A fairness checklist — covering optimisation level, clock frequency, memory configuration, RTOS wrapper differences, measurement instrumentation overhead, and environmental differences — is applied to every comparison. Any fairness-sensitive difference is reported *before* performance conclusions.
- **Provenance tracking.** Every acquired resource (firmware, CMSIS headers, BSP files, documentation) is recorded with its source URL, version, branch/tag/commit, and acquisition date.
- **Command transparency.** Every command the agent executes (build, debug, inspection) is reported with its exact invocation, working directory, and outcome.

These rules solve a specific problem with using general-purpose LLMs for embedded work: without them, a model may silently simplify a benchmark, invent an API that does not exist, claim that code works when it has never been compiled, or obscure the provenance of acquired materials. The always-on policy prevents all four failure modes.

3.7 Semantic preservation in detail

The semantic-equivalence skill deserves elaboration because it is the mechanism that keeps cross-vendor comparisons valid. When the Creation-Porting agent produces a target project, the semantic-equivalence skill is invoked to compare the target against the reference on every point of the checklist. The result is classified into one of five categories:

1. **Equivalent** — behaviour matches within the benchmark definition.
2. **Equivalent with caveats** — behaviour matches, but environment or implementation details affect fairness (e.g. a different tick source).
3. **Drift** — behaviour differs and may affect results.
4. **Approval required** — an intentional deviation that the user must accept.
5. **Unknown** — evidence is insufficient to determine equivalence.

Any result other than category 1 halts the workflow and produces a clear report of what differs and why. This prevents semantic drift from silently propagating into published benchmark numbers.

3.8 Deterministic campaign workflow

Every benchmark campaign is intended to follow the same twelve-step sequence. This stable ordering makes the expected inputs, decisions, approval gate, and outputs explicit. It supports consistent execution and later audit, but does not by itself prove that repeated agent sessions will produce identical projects or measurements.

1. **Resolve scenario** — identify benchmark, board, MCU, and firmware from partial user input and workspace context.
2. **Acquire materials** — fetch missing firmware, startup, linker, CMSIS, BSP, and device descriptions from official sources; record provenance.
3. **Analyse semantics** — extract exact observable behaviours, identify portable versus board-specific layers (invoke *benchmark-analysis* skill).
4. **Resolve IAR template** — look up the target board in the template registry; acquire and materialise if needed (invoke *iar-template-resolution* skill).

5. **Create or port project** — delegate to Creation-Porting agent; generate new target project at the application layer only.
6. **Configure build** — delegate to Build-Project agent; set device, debugger, startup, linker, defines, includes consistently.
7. **Build** — execute IarBuild.exe autonomously; classify failures; fix and rebuild.
8. **Debug preparation** — delegate to Debugger agent; parse logs, apply minimal fixes, revalidate.
9. **Hardware approval gate** — ask user before flashing/attaching to real board.
10. **Extract measurements** — capture runtime metrics from UART, debugger, or batch sessions.
11. **Semantic equivalence and fairness review** — verify that the port preserves all benchmark-observable behaviours; report all fairness-sensitive differences.
12. **Generate report** — produce structured output with provenance, results, fairness status, and unresolved unknowns.

Figure 3.3 shows the end-to-end lifecycle with the handoff point to MCU Workflow Studio.

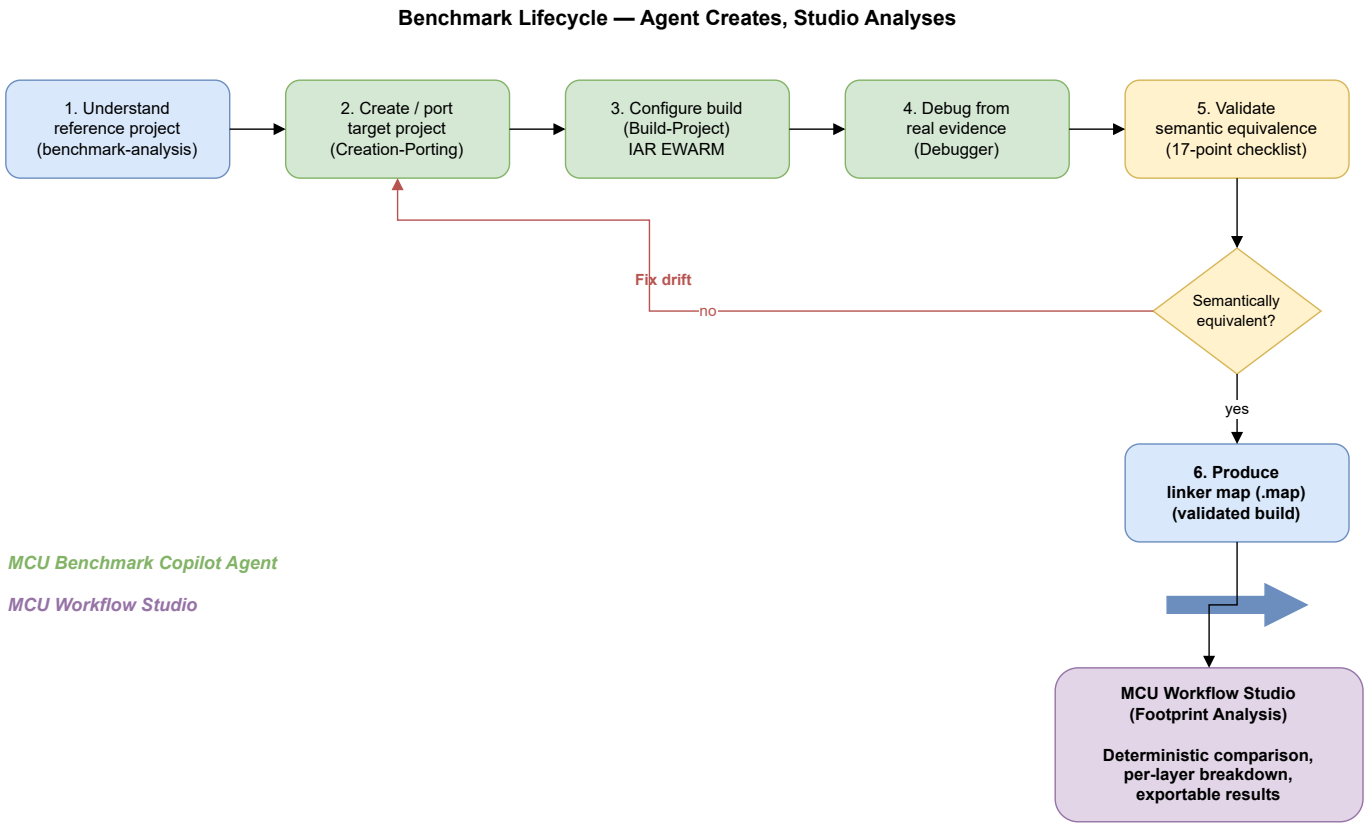

Figure 3.3: End-to-end benchmark lifecycle. The MCU Benchmark Copilot Agent handles steps 1 through 11 autonomously (with a hardware gate at step 9); once semantic equivalence is confirmed, the validated linker map is handed to MCU Workflow Studio for footprint analysis.

Figure 3.4 presents the same campaign as a UML sequence diagram, showing the actual participant delegation, message flow, the hardware approval gate, and artifact emission at each step.

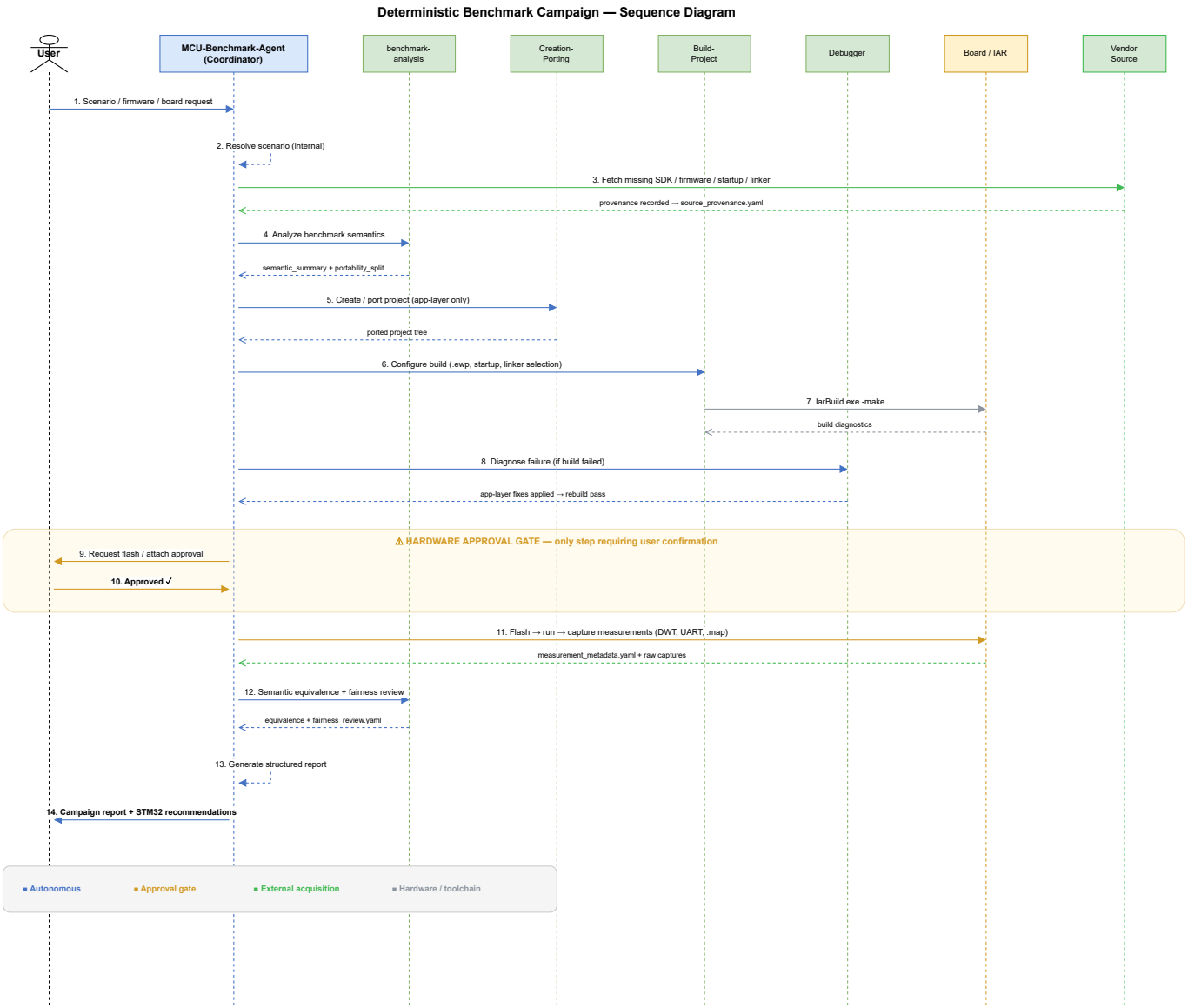

Figure 3.4: UML sequence diagram of a deterministic benchmark campaign. The coordinator delegates to specialist agents (benchmark-analysis, Creation-Porting, Build-Project, Debugger) and interacts with external actors (Vendor Source, Board/IAR). The hardware approval gate (steps 9–10, highlighted) is the only point requiring user confirmation; all other steps execute autonomously.

This separation keeps each tool focused. The agent does not attempt footprint analysis, and the studio does not attempt code generation or debugging. Together, they cover the full lifecycle of a cross-vendor benchmark: resolve, acquire, create, build, debug, validate, measure, then analyse.

3.9 IAR template resolution

Before creating any benchmark project, the system must resolve the correct IAR project template for the target board. We designed a deterministic four-step resolution algorithm encoded in the *iar-template-resolution* skill:

1. **Exact board match** — search the template registry (`iar_templates.yaml`) for an entry whose `board_id` matches the target exactly.
2. **Family fallback** — if no exact match, search for a family-level fallback template and adjust device, startup, and linker for the specific MCU.
3. **Board-type fallback** — if no family match, select the closest template matching the board type and vendor with explicitly downgraded confidence.
4. **Unresolved** — if no suitable template exists, document the gap and report to the user rather than fabricating a template.

When a template is identified but not yet materialised locally, the `acquire_iar_template.ps1` script fetches it from official vendor GitHub repositories or SDK packages, records provenance, and verifies the file inventory. The materialised template then serves as the read-only project baseline into which benchmark application code is integrated through designated integration points.

3.10 Source acquisition policy

When materials are missing, the agent acquires them autonomously using a strict preference order:

1. Workspace files already present.
2. Local SDK content (IAR device/config directories, installed CMSIS packs).
3. Official vendor GitHub repositories (tagged releases preferred).
4. Official vendor websites, SDK packages, CMSIS Pack Index.
5. Third-party mirrors — only as a last resort, with an explicit risk note.

For every acquired resource, the agent records: source URL, version or branch/tag/commit, package name, and date of acquisition. This provenance chain ensures that any benchmark result can be traced back to the exact firmware version used, satisfying the reproducibility requirements of academic evaluation.

3.11 Campaign artifact model

Each benchmark campaign persists its state in a structured dossier under `.github/benchmark/campaigns/<campaign_id>/`. Key artifacts include:

- `campaign_manifest.yaml` — campaign identity, platforms, and stage tracking.
- `execution_status.yaml` — per-stage status with timestamps and blockers.
- `source_provenance.yaml` — acquisition provenance for every resource used.
- `scenario_resolution.yaml` — how partial input was resolved.
- `fairness_baseline.yaml` — per-platform configuration baseline for fairness comparison.
- `semantic_equivalence.yaml` — verification results per checklist item.
- `fairness_review.yaml` — all fairness-sensitive differences identified.
- `measurement_metadata.yaml` — metrics collected, methods, triggers, and confidence levels.

This artifact model makes every campaign auditable and reproducible. A reviewer can inspect the dossier to understand exactly what was resolved, acquired, built, measured, and concluded — without re-running the campaign.

3.12 Prompt-driven usage

To lower the barrier to entry, the system ships ten pre-built prompt templates (`.github/prompts/`) that encode common benchmark tasks. The user selects a prompt from the Copilot chat picker and fills in minimal parameters (a board name, a scenario, or a firmware package name); the agent handles everything else. Table 3.7 lists the available prompts.

Table 3.7: Pre-built prompt templates for common benchmark tasks.

#	**Purpose**
01	Autonomous benchmark from firmware name only — agent resolves, acquires, builds, and measures.
02	Port a benchmark to another board with fairness guarantee.
03	Build, debug, and measure an existing project autonomously.
04	Compare two boards fairly with semantic-equivalence check.
05	Recommend STM32 improvements after a measured competitor gap.
06	Create a new benchmark from a scenario description (no reference).
07	Port STM32 reference to a competitor platform.
08	Port competitor benchmark to STM32.
09	Port between two non-ST vendors.
10	Resolve and build from a firmware/SDK package name alone.

3.13 Implementation

The entire agent system is implemented within a `.github/` directory, leveraging the VS Code GitHub Copilot extensibility framework [16, 23, 24]. Agent and skill definitions use YAML frontmatter to declare metadata (name, description, tool permissions) and Markdown bodies for behavioural instructions. The automation tools are standalone PowerShell and Python scripts. Table 3.8 summarises the file structure.

Table 3.8: File structure of the agent system.

Path	Role
`.github/copilot-instructions.md`	Always-on rules inherited by every agent and skill.
`.github/agents/*.agent.md`	One file per specialist agent (4 files).
`.github/skills/*/SKILL.md`	One directory per skill, each containing a structured checklist (6 directories).
`.github/tools/*.ps1, *.py`	Twelve automation scripts for deterministic operations.
`.github/config/*.yaml`	Eight configuration registry files (boards, MCUs, vendors, toolchains, templates, sources, measurements, reporting).
`.github/prompts/*.prompt.md`	Ten pre-built prompt templates for common tasks.
`.github/templates/benchmark-dossier/`	Campaign artifact templates (YAML skeletons).
`.github/docs/`	Developer documentation (quickstart, porting guide, validation guide, naming conventions, etc.).
`.github/schemas/`	JSON/YAML schemas for artifact validation.

The implementation requires no external server, no API keys beyond the user's existing GitHub Copilot subscription, and no deployment pipeline. Adding a new agent, skill, tool, or board entry is a matter of creating a file and restarting the Copilot Chat session. This minimal footprint was deliberate: the system should be as easy to maintain and version-control as the benchmark source code itself.

3.14 Integration with the benchmarking workflow

The agent system occupies a precise position in the benchmarking pipeline. It sits *before* MCU Workflow Studio in the workflow: the agent resolves the scenario, acquires materials, creates and

validates the target project, ensures it compiles under IAR EWARM, confirms that its runtime behaviour matches the reference, and extracts measurements. Only then does the build's linker map become an input to MCU Workflow Studio for footprint analysis. The relationship is strictly sequential and complementary — the agent is responsible for *producing and validating* correct builds, and the studio is responsible for *measuring and comparing* them.

3.15 Engineering value and limitations

The agent system encodes the benchmark methodology in reusable policies, checklists, specialist roles, and structured campaign artifacts. This design makes semantic equivalence, fairness, provenance, baseline immutability, and unresolved unknowns explicit during a campaign. It can assist with source discovery, project preparation, build execution, and diagnostic interpretation while preserving a hardware-approval gate before operations on a physical board.

The structured outputs make completed actions and remaining blockers easier to inspect. However, this report does not contain a controlled productivity study comparing agent-assisted and manual porting, nor does it demonstrate that every campaign can proceed without human intervention. We therefore do not quantify time savings or claim autonomous completion as a general result. Reproducibility also depends on the model version, available tools, source packages, configuration state, and hardware observations; the campaign dossier improves traceability but cannot remove these dependencies.

The structured output contracts — each agent returns objective, scenario resolution, actions executed, semantic equivalence status, fairness status, key findings, unresolved unknowns, and next action — make the work transparent and reviewable without re-executing the campaign.

We acknowledge several limitations. The system depends on the quality of the underlying language model; while the always-on rules and the configuration registry constrain its behaviour, they cannot prevent all model errors, particularly for vendor APIs that are poorly represented in the model's training data. The tool supports IAR EWARM 9.60.3 as the primary toolchain; extending it to other compilers (GCC, Keil) would require additional Build-Project configurations and toolchain registry entries. The evidence-driven validation policy means the system halts at the hardware gate — it cannot flash a board without explicit approval. This is by design: in embedded benchmarking, the hardware is the ground truth, and no amount of static reasoning can substitute for it.

Conclusion

We designed and built the MCU Benchmark Copilot Agent as a structured assistant for benchmark campaigns. Its specialist agents, reusable skills, automation scripts, and configuration registry organise the path from scenario resolution to evidence review and reduce unsupported hardware assumptions. Shared policies expose semantic-equivalence checks, fairness risks, baseline changes, provenance, and unresolved unknowns rather than treating an unverified build as a completed benchmark.

Together with MCU Workflow Studio, the agent supports two complementary parts of the method: project preparation and traceability on one side, and linker-map footprint analysis on the other. The system records information needed for later reproduction, but its productivity, repeatability, and end-to-end autonomy still require evaluation through recorded builds and hardware-backed benchmark runs.

Chapter 4

Benchmarking ST against GD32

Introduction

GigaDevice Semiconductor is a Chinese fabless semiconductor company that produces the GD32 series of 32-bit microcontrollers [26]. The GD32 family is often positioned as a pin-compatible, drop-in alternative to STM32 parts, sharing the same Arm Cortex-M cores, similar peripheral sets, and comparable pricing. This close correspondence makes GD32 a particularly relevant competitor: engineers evaluating an MCU platform frequently shortlist both vendors, and the choice often comes down to measurable differences in software-stack quality rather than silicon specifications.

In this chapter we compare the STM32Cube ecosystem against GigaDevice's GD32 firmware libraries on two middleware stacks: the **USB device stack** (exercised through HID and CDC scenarios) and the **FreeRTOS** real-time operating system integration (exercised through basic, co-operative, and preemptive scheduling scenarios). For each stack we present the scenario design, per-scenario footprint and execution-time results, a per-layer ROM breakdown, and an interpretation of the trade-offs the data reveals.

All measurements follow the methodology described in Chapter 1: identical scenarios on both platforms, same compiler optimisation level, same clock frequency, and results extracted from IAR EWARM linker maps using the footprint analysis tool presented in Chapter 2.

4.1 Hardware platforms

Every scenario in this chapter runs on two boards hosting comparable Cortex-M33 parts: the STMicroelectronics **NUCLEO-U575ZI-Q** (STM32U575ZIT6Q) and GigaDevice's **GD32E507** evaluation board (GD32E507ZET6). Both devices integrate an Arm Cortex-M33 core clocked at 160 MHz for these tests, ensuring that performance differences reflect software-stack efficiency rather than raw silicon speed. Table 4.1 contrasts their key characteristics.

Table 4.1: Board characteristics: ST reference vs GigaDevice GD32.

Characteristic	**ST (STM32U575ZIT6Q)**	**GD32 (GD32E507ZET6)**
Core	Arm Cortex-M33	Arm Cortex-M33
Clock (test)	160 MHz	160 MHz
Flash	2 MB	512 KB
SRAM	786 KB	128 KB
USB	Full-Speed device	Full-Speed device
Security	TrustZone, AES, PKA, HASH	—
Toolchain / IDE	STM32CubeIDE / IAR EWARM	IAR EWARM

The STM32U575 belongs to the ultra-low-power STM32U5 series and offers substantially more flash and RAM, hardware security accelerators, and a richer peripheral set [28]. The GD32E507 targets a similar performance point at a competitive price but with fewer on-chip resources [27]. For our benchmarks, the relevant constant is that *both devices run the same core at the same frequency*, so footprint and execution-time differences are attributable to the vendor's software stack and HAL implementation.

4.2 USB device stack

The Universal Serial Bus (USB) device stack is a critical middleware component for any MCU that needs to communicate with a host PC. We benchmark ST's USB Device Library (part of the STM32Cube middleware) against GigaDevice's USB library, both operating in **Full-Speed** mode and running bare-metal (no RTOS).

4.2.1 Scenario design

We defined two scenarios that cover the most commonly used USB device classes:

1. **HID (Human Interface Device)** — a low-rate interrupt-transfer scenario in which the MCU acts as a USB mouse, sending periodic HID reports to the host. This exercises the enumeration path, descriptor handling, and interrupt-endpoint transfer logic.

2. **CDC (Communication Device Class)** — a higher-throughput bulk-transfer scenario in which the MCU exposes a virtual COM port. The host sends data to the device, which echoes it back. This exercises bulk IN/OUT transfers, line-coding negotiation, and buffering.

Each scenario was implemented identically on both platforms: same USB descriptors, same endpoint configuration (Full-Speed), same clock source (internal oscillator — HSI48 on STM32, IRC48M on GD32), and same core frequency of 160 MHz.

Table 4.2 summarises the scenarios and the metrics collected.

Table 4.2: USB device stack benchmark scenarios.

Scenario	**What it exercises**	**Metrics**
HID	Interrupt transfers, enumeration, descriptor handling	ROM/RAM (bytes), init/enum/TX time (ms)
CDC	Bulk transfers, virtual COM port, echo loopback	ROM/RAM (bytes), init/enum/TX/RX time (ms)

4.2.2 Results

4.2.2.1 HID scenario

Table 4.3 presents the total ROM and RAM footprint for the HID scenario.

Table 4.3: USB HID device footprint (bytes).

Metric	**STM32U575**	**GD32E507**	**Difference**
ROM (total)	16 269	9 175	+77 %
RAM (total)	3 446	9 608	−65 %

The STM32 firmware consumes 77 % more ROM than GD32, but uses 65 % *less* RAM. The reported whole-image totals differ by 6 162 bytes. The available report artifacts do not provide a reconciled stack, heap, and static-data breakdown. In particular, the previously recorded GD32 allocations of 8 KB for both stack and heap would already exceed the reported total RAM footprint. We therefore retain the measured totals but do not attribute the difference to stack or heap configuration pending verification against the original linker maps.

Table 4.4 breaks down the ROM consumption by architectural layer.

Table 4.4: USB HID ROM breakdown by layer (bytes).

Layer	**STM32U575**	**GD32E507**	**Difference**
Application	2 580	1 942	+33 %
HAL drivers	10 426	820	+1 171 %
MW stack	2 859	5 743	−50 %
Classified subtotal	15 865	8 505	—
Other/unattributed	404	670	—
Total ROM	16 269	9 175	+77 %

The "Other/unattributed" row is the arithmetic residual between the classified layer subtotal and the reported whole-image ROM total. Its module composition cannot be recovered from the available report artifacts.

The classification associates most of the difference with the HAL layer. On STM32, the HAL RCC module accounts for 4 560 bytes (44 % of total HAL ROM) and the combined HAL/LL USB driver adds another 4 988 bytes (48 %). GD32's classified HAL layer is 820 bytes, while its classified USB middleware ROM (5 743 bytes) is twice the size of ST's (2 859 bytes). This distribution is consistent with different layer boundaries, but the available artifacts do not establish the design decision that produced them.

The classified STM32 application layer is 33 % larger. User configuration files (`usbd_desc.c`, `usbd_conf.c`) account for 985 bytes, or 38 % of its application ROM. This attribution identifies where the bytes occur; it does not by itself measure configuration flexibility or establish why the vendor architectures differ.

Table 4.5 shows the execution-time measurements.

Table 4.5: USB HID execution time (ms).

Operation	**STM32U575**	**GD32E507**	**Difference**
Initialisation	9.990	26.084	−61.7 %
Enumeration	213	216	−1.4 %
Data transmission	0.0093	0.0094	−0.7 %

Under the reported test configuration, STM32 initialisation took 9.990 ms and GD32 initialisation took 26.084 ms, a 61.7 % difference. Enumeration and transmission differed by 1.4 % and 0.7 %, respectively. The available evidence does not isolate clock configuration, peripheral bring-up, host scheduling, or USB protocol timing, so these measurements support an observed difference but not

a specific causal explanation.

4.2.2.2 CDC scenario

Table 4.6 presents the total footprint for the CDC scenario.

Table 4.6: USB CDC device footprint (bytes).

Metric	**STM32U575**	**GD32E507**	**Difference**
ROM (total)	17 049	9 085	+88 %
RAM (total)	4 089	9 683	−57.7 %

The cross-vendor pattern mirrors HID: STM32 uses more ROM but less RAM than GD32. Relative to HID, the STM32 CDC image adds 780 bytes of ROM and 643 bytes of RAM. The GD32 CDC image uses 90 bytes less ROM and 75 bytes more RAM than its HID image. The available layer totals do not establish which CDC mechanisms cause these changes.

Table 4.7 provides the per-layer ROM breakdown.

Table 4.7: USB CDC ROM breakdown by layer (bytes).

Layer	**STM32U575**	**GD32E507**	**Difference**
Application	2 683	1 882	+42.5 %
HAL drivers	10 446	820	+1 174 %
MW stack	2 423	5 709	−57.5 %
Classified subtotal	15 552	8 411	—
Other/unattributed	1 497	674	—
Total ROM	17 049	9 085	+88 %

The residual row again reconciles the classified subtotal with the whole-image total. The available report artifacts do not identify whether the residual comprises startup code, read-only data, unclassified modules, or another accounting category.

The classified distribution is similar to HID. The STM32 HAL RCC module (4 560 bytes) and the HAL/LL USB driver (5 006 bytes) together account for 92 % of the HAL ROM. The STM32 application layer is 42.5 % larger; the USB device configuration files (`usb_device.c`, `usbd_cdc_if.c`, `usbd_desc.c`, `usbd_conf.c`), contributing 1 164 bytes (43 % of application ROM). The classified STM32 USB middleware is 57.5 % smaller than GD32's. These figures locate the difference but do not establish a causal architectural trade-off.

Table 4.8 shows the execution-time measurements.

Table 4.8: USB CDC execution time (ms).

Operation	**STM32U575**	**GD32E507**	**Difference**
Initialisation	9.994	26.076	−61.7 %
Enumeration	218.2	218.5	−0.1 %
Data transmission	0.00988	0.01003	−1.5 %
Data reception	0.00986	0.00993	−0.7 %

The CDC results reproduce the reported initialisation difference: 9.994 ms for STM32 and 26.076 ms for GD32. Enumeration, transmission, and reception differ by no more than 1.5 % in the reported observations. Without repeated samples and controlled host-side timing, these small differences should be treated as near-equivalent observations rather than evidence that either implementation is protocol-bound or software-bound.

4.2.3 Interpretation

Measurement controls and limitations

The available artifacts do not record the host computer and operating system, USB host controller, cable, host application, endpoint packet sizes and polling intervals, payload length, transfer count, timer source, timing boundary, warm-up policy, repetition count, or dispersion. Exact compiler and linker options and evidence that descriptors and endpoint configurations are byte-equivalent are also unavailable. The timing values are therefore retained as reported observations, but causal and protocol-level conclusions remain unresolved.

The USB benchmark reveals a clear architectural trade-off between the two vendors.

ROM footprint

STM32 consumes 77–88 % more total ROM than GD32 across both scenarios. The dominant contributor is the HAL layer, which is roughly twelve times larger on STM32. Two modules drive this cost: the **HAL RCC** clock-configuration driver (≈4 560 bytes) and the **HAL/LL USB** peripheral driver (≈5 000 bytes). GigaDevice achieves a much smaller HAL by embedding peripheral-initialisation logic inside the middleware library rather than exposing it as a separate, reusable driver layer.

RAM footprint

The reported whole-image RAM totals favour STM32 by 57–65 % in the two USB scenarios. No reconciled allocation breakdown is available, so the difference cannot currently be attributed to dynamic allocation, static buffers, call-stack depth, or heap configuration.

Execution time

The reported initialisation observations favour STM32 by 61.7 % in both HID and CDC. After initialisation, the reported values differ by no more than 1.5 %. Repeated samples and a documented timing boundary are required before assigning these differences to the software stacks or the USB protocol.

Design philosophy

STM32Cube separates concerns: HAL handles clocks and peripheral access, the middleware handles protocol logic, and the application owns configuration through user-editable files (`usbd_conf.c`, `usbd_desc.c`). This modular design gives developers explicit control and reusability at the cost of a larger total ROM. GigaDevice takes a more monolithic approach — smaller HAL, but a thicker middleware that bundles configuration internally, reducing ROM but also reducing the user's configuration flexibility.

Enhancement proposals

The data suggests two concrete areas where STM32Cube could reduce ROM without sacrificing functionality:

1. **HAL RCC optimisation.** The HAL RCC module accounts for ≈4.5 KB in every USB project regardless of the clocks actually configured. A conditionally compiled or link-time-optimised RCC driver that includes only the clock paths used would significantly narrow the HAL gap.

2. **HAL/LL USB driver consolidation.** The combined HAL and LL USB driver (≈5 KB) provides a comprehensive abstraction but may include code paths (e.g. High-Speed, OTG, DMA) that Full-Speed-only projects never execute. Finer compile-time selection of USB features could reduce this cost.

4.3 FreeRTOS integration

FreeRTOS is the most widely adopted real-time operating system for Cortex-M microcontrollers [29]. Both STMicroelectronics and GigaDevice ship FreeRTOS integration packages as part of their

firmware offerings. We benchmark the two implementations using FreeRTOS v11.2 [30], running on the same two boards at 160 MHz with a 1 ms tick (time slice) per task.

4.3.1 Scenario design

We defined three scheduling scenarios that stress progressively more complex RTOS mechanisms:

1. **Basic processing** — a single counting thread runs at low priority alongside a high-priority reporting thread. The counting thread increments a global counter in a tight loop; the reporting thread wakes every 30 s and prints the counter delta. This isolates single-task throughput with minimal scheduler overhead.

2. **Cooperative processing** — five threads at the same low priority, each incrementing its own counter and then calling `taskYIELD()` to voluntarily yield the processor. A high-priority reporter prints all five counters every 30 s. This exercises voluntary context switching under equal-priority round-robin scheduling.

3. **Preemptive processing** — five threads at ascending priorities. Each thread resumes the next-higher-priority thread (which preempts it), increments its counter, then suspends itself. A high-priority reporter prints all five counters every 30 s. This exercises the full preemptive scheduler: suspend, resume, and priority-based preemption.

Table 4.9 summarises the three scenarios.

Table 4.9: FreeRTOS benchmark scenarios.

Scenario	**What it exercises**	**Metric**
Basic	Single-thread throughput, minimal scheduler overhead	Thread iterations/s
Cooperative	Voluntary context switching (`taskYIELD`)	Total thread iterations/30 s
Preemptive	Priority-based preemption, suspend/resume	Total thread iterations/30 s

4.3.2 Results

4.3.2.1 Basic processing

Table 4.10 shows the performance result and Table 4.11 the footprint.

Table 4.10: FreeRTOS basic processing — performance.

Metric	**STM32U575**	**GD32E507**	**Difference**
Thread iterations/s	47 943	47 955	−0.025 %

Table 4.11: FreeRTOS basic processing — footprint (bytes).

Metric	**STM32U575**	**GD32E507**	**Difference**
ROM	14 266	10 128	+40.9 %
RAM	11 960	11 716	+2.1 %

The reported iteration rates differ by 12 iterations/s, or 0.025 %, and are therefore nearly equal for this observation. Without repeated measurements and dispersion data, the result does not establish a difference in scheduler efficiency. STM32 uses 40.9 % more ROM and 2.1 % more RAM in the reported images.

4.3.2.2 Cooperative processing

Table 4.12: FreeRTOS cooperative processing — performance.

Metric	**STM32U575**	**GD32E507**	**Difference**
Thread iterations/30 s	1 220 066	1 200 666	+1.6 %

Table 4.13: FreeRTOS cooperative processing — footprint (bytes).

Metric	**STM32U575**	**GD32E507**	**Difference**
ROM	15 422	11 224	+37 %
RAM	10 972	10 728	+2.3 %

STM32 reports 1.6 % more iterations over the 30-second observation window. This result includes the complete worker loop and scheduling environment; it does not isolate the execution cost of `taskYIELD()`. The footprint gap is 37 % ROM.

4.3.2.3 Preemptive processing

Table 4.14: FreeRTOS preemptive processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/30 s	294 615	285 144	+3.3 %

Table 4.15: FreeRTOS preemptive processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	16 226	12 028	+35 %
RAM	10 996	10 748	+2.3 %

Preemptive scheduling is the most demanding scenario, involving suspend, resume, and priority-based task switching. STM32 achieves 3.3 % more iterations, the largest performance advantage across the three FreeRTOS scenarios. The ROM gap (+35 %) is consistent with the other scenarios.

4.3.2.4 Per-layer ROM breakdown

To understand *where* the ROM overhead originates, Table 4.16 breaks down the ROM footprint into three architectural layers — application, HAL drivers, and FreeRTOS middleware stack — for all three scenarios.

Table 4.16: FreeRTOS ROM breakdown by layer (bytes).

Scenario	Layer	STM32U575	GD32E507	Diff.
Basic	Application	4 531	4 278	+6 %
	HAL drivers	4 927	1 078	+357 %
	MW stack	4 808	4 772	+0.7 %
Cooperative	Application	5 823	5 484	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	4 688	4 662	+0.6 %
Preemptive	Application	6 191	5 810	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	5 124	5 140	−0.3 %

The pattern is strikingly consistent across all three scenarios:

- The **FreeRTOS middleware stack** itself is virtually identical in size (≤1 % difference). Both vendors ship the same upstream FreeRTOS kernel, so this is expected.
- The **application layer** is ≈6 % larger on STM32, reflecting slightly more initialisation code in the STM32Cube project template.
- The **HAL layer** is 355–357 % larger on STM32, entirely accounting for the total ROM gap. The dominant contributor is the **HAL RCC** module, which configures the clock tree.

4.3.3 Interpretation

Measurement controls and limitations

The available evidence does not establish identical FreeRTOS configuration macros, port layer and wrapper, compiler options, interrupt load, task stack sizes, tick source, reporting overhead, warm-up policy, repetition count, or result dispersion. The 30-second counter values compare complete scenario executions; they are not direct measurements of context-switch latency.

Performance

The reported observations are nearly equal in the basic scenario and favour STM32 by 1.6 % and 3.3 % in the cooperative and preemptive scenarios. These aggregate iteration counts do not isolate context-switch latency or establish that the difference grows because of scheduler complexity. Repeated measurements and a direct cycle-level context-switch measurement are required before attributing the observations to either FreeRTOS port.

ROM footprint

The ROM overhead follows the same pattern as the USB stack: a 35–41 % total increase driven almost entirely by the HAL layer, specifically the HAL RCC clock-configuration module. Crucially, the FreeRTOS middleware itself contributes no measurable overhead — the gap is in the vendor's hardware-abstraction code, not in the RTOS kernel.

RAM footprint

RAM differences are negligible (2.1–2.3 %), indicating that both FreeRTOS ports allocate task stacks and kernel structures similarly.

Enhancement proposals

The FreeRTOS results reinforce the conclusion from the USB benchmark: the HAL RCC module is the single largest contributor to STM32's ROM overhead across *all* middleware stacks. A targeted effort to slim the RCC driver — through dead-code elimination, conditional compilation based on the clocks actually used, or link-time optimisation — would benefit every STM32 project, not just FreeRTOS.

Conclusion

The ST-versus-GD32 comparison across two middleware stacks (USB device and FreeRTOS) and five scenarios reveals a consistent pattern:

- **Performance:** the reported observations favour STM32 for USB initialisation and by up to 3.3 % in the FreeRTOS scenarios, while steady-state USB values are close. The absence of repeated samples and dispersion limits stronger claims.
- **ROM footprint:** STM32 consumes 35–88 % more flash, with the HAL layer (principally HAL RCC and, for USB, HAL/LL USB) responsible for the bulk of the difference. The middleware stacks themselves are comparable in size.
- **RAM footprint:** STM32 uses significantly less RAM than GD32 in USB scenarios (57–65 % less) thanks to smaller default stack/heap allocations and more static memory management. FreeRTOS RAM is nearly identical.
- **Architectural trade-off:** STM32Cube's modular, layered design (separate HAL, middleware, and user-configuration files) provides greater flexibility and reusability but at a measurable ROM cost. GigaDevice's more monolithic approach yields smaller ROM by bundling driver and configuration logic into the middleware, at the expense of less user-visible modularity.

The single most impactful enhancement for STM32 would be **optimising the HAL RCC module**: it appears in every project and consistently accounts for the largest share of the HAL ROM gap. A secondary improvement for USB projects would be **finer-grained compile-time selection** of USB driver features (Full-Speed vs High-Speed, device vs OTG) to exclude unused code paths.

Chapter 5

Benchmarking ST against Texas Instruments

Introduction

Texas Instruments (TI) is one of the world's largest semiconductor manufacturers, with a broad portfolio of embedded processors and microcontrollers [31]. Its MSPM0 and MSPM3 families, built around Arm Cortex-M0+ and Cortex-M33 cores respectively, target cost-sensitive and mixed-signal applications with a strong emphasis on analog integration [32]. Unlike GigaDevice, which competes primarily on pin-compatibility, TI competes on a fundamentally different SDK architecture: a SysConfig-centric, unified serial peripheral model and a single-layer driver library (driverlib) rather than the dual HAL/LL approach used by STMicroelectronics.

This chapter presents the ST-versus-TI comparison in two complementary phases. The first is a comprehensive **static analysis** of the low-level driver layers of both SDKs. Unlike the runtime-focused benchmarks in Chapter 4, this analysis examines source-code quality and SDK breadth without executing firmware on hardware. The second phase combines twelve low-level-driver scenarios with a FreeRTOS middleware benchmark covering basic, cooperative, and preemptive processing. We compare the STM32CubeC5 Low-Layer (LL) drivers [39] against the TI MSPM33 SDK driverlib [34] across seven quality dimensions — lines of code, documentation coverage, code duplication, cyclomatic complexity, API surface design, MISRA C:2012 compliance, and security (CWE analysis). For the scope definition, we also map the SDK coverage in terms of peripheral drivers, middleware, cryptographic algorithms, and communication protocols, identifying what each vendor offers exclusively.

All measurements were produced by automated scripts using `cloc` [43], `cppcheck` [44] (with its `-addon=misra` and `-addon=cert` modes), `lizard` [45] for cyclomatic complexity, and `jscpd` [46] for duplication detection.

5.1 Hardware platforms

The analysis in this chapter targets driver code for two boards hosting comparable Cortex-M33 parts. Table 5.1 summarises their key characteristics.

Table 5.1: Board and MCU comparison — STM32C5A3 vs. MSPM33C321A.

Feature	**STM32C5A3 (NUCLEO-C5A3ZG)**	**MSPM33C321A (LP-MSPM33C321A)**
Core	Arm Cortex-M33 (FPU, DSP, MPU)	Arm Cortex-M33 (FPU, DSP, TrustZone)
Max frequency	144 MHz	160 MHz
Flash	1 MB (ECC, dual-bank)	1 MB (ECC, dual-bank, address swap)
SRAM	256 KB (128 KB with ECC)	256 KB (ECC)
ADC	3 × 12-bit, 4.5 MSPS	2 × 12-bit, 9.4 MSPS
DAC	1 × 12-bit	2 × 8-bit
CAN	2 × FDCAN	2 × CAN FD
USB	USB 2.0 FS (Host/Device/DRD)	—
Ethernet	Yes (MAC with DMA)	—
I3C	Yes	—
Debug probe	ST-LINK	XDS110 (EnergyTrace)

Both parts share the same core architecture and identical flash/RAM capacities, which makes the comparison fair at the silicon level. The STM32C5A3 offers broader connectivity (USB, Ethernet, I3C), while the MSPM33C321A favours higher analog bandwidth (9.4 MSPS ADC) and provides TrustZone support [33, 40].

5.2 Static analysis of the driver layer

This section compares the LL driver layer of STM32CubeC5 against the driverlib of the MSPM33 SDK. The LL layer was chosen for ST because it is the closest architectural equivalent to TI's driverlib: both are thin, inline-heavy abstractions over peripheral registers, designed for performance-critical code. We analyse seven complementary dimensions.

5.2.1 Scope and methodology

The analysis targets the following source sets:

- **ST:** all 33 header files in `stm32c5xx_drivers/ll/` (the LL layer of STM32CubeC5).
- **TI:** all 85 source and header files in `source/ti/driverlib/` and its `m33/` subdirectory (the driverlib of MSPM33 SDK v1.03.00.01).

Both sets were analysed on the same workstation under identical tool configurations. The tools and their roles are:

- **cloc** v2.08 [43] — counts blank, comment, and code lines per language and file.
- **lizard** [45] — computes cyclomatic complexity, number of parameters, and token count per function.
- **cppcheck** v2.x [44] — runs MISRA C:2012 addon (`-addon=misra`) and CWE/CERT checks.
- **jscpd** [46] — detects copy-pasted code blocks (duplication).
- A custom Python analysis script extracts API surface metrics (public/static/inline functions, macros, enums, naming patterns, parameter distributions) and documentation coverage (Doxygen blocks, `@brief`, `@param`, `@return` tags, file headers).

5.2.2 Lines of code

Table 5.2 presents the line-count breakdown for each SDK's driver layer.

Table 5.2: Lines-of-code comparison — ST LL vs. TI driverlib.

Metric	ST (LL)	TI (driverlib)
Source files	33	85
Blank lines	5 744	8 747
Comment lines	57 649	45 763
Code lines	22 292	27 972
Total lines	85 685	82 482
Comment-to-code ratio	2.59	1.64

ST delivers its LL layer in 33 header-only files containing 22 292 lines of code, while TI spreads its driverlib across 85 files (both `.c` and `.h`) with 27 972 code lines. Despite having 25% more code, TI's total line count is slightly lower because ST invests significantly more in inline comments: 57 649 comment lines versus 45 763, yielding a comment-to-code ratio of 2.59 for ST compared with 1.64 for TI. This difference reflects ST's header-only, fully-documented inline function strategy, where each function carries extensive Doxygen annotations embedded alongside the implementation.

5.2.3 Documentation coverage

We measured documentation quality through Doxygen tag extraction. Table 5.3 summarises the results.

Table 5.3: Documentation coverage — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Total functions	3 713	3 056
Documented functions	3 613	2 525
Doc. coverage	**97.3 %**	**82.6 %**
Doxygen blocks	5 967	3 357
`@brief` tags	3 826	5 568
`@param` tags	4 440	3 970
`@return` tags	1 705	2 172
File headers	33	85
Inline comments	0	283

ST achieves 97.3% function-level documentation coverage, leaving only 100 undocumented functions. TI reaches 82.6%, with 531 undocumented functions — mostly in `.c` implementation files, where TI concentrates complex logic (e.g. flash controller operations, PLL configuration) without per-function Doxygen blocks. This architectural choice is deliberate: TI documents the *header* API thoroughly but omits Doxygen in `.c` bodies, whereas ST's header-only approach means every function definition is also its documented declaration.

Interestingly, TI uses more `@brief` tags (5 568 vs. 3 826) and more `@return` tags (2 172 vs. 1 705), suggesting that when TI does document a function, it tends to be descriptive about purpose and return values. ST, however, produces 78% more Doxygen blocks overall (5 967 vs. 3 357), reflecting its higher raw volume of documented entities.

5.2.4 Code duplication

Table 5.4 presents the duplication metrics.

Table 5.4: Code duplication — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Total lines analysed	85 685	82 403
Code lines	10 461	18 008
Duplicate blocks	71	414
Duplicate lines	229	1 306
Duplication %	**2.19 %**	**7.25 %**
Same-file clones	71	338
Cross-file clones	0	76

TI's driver layer exhibits 3.3× more duplication than ST's (7.25% vs. 2.19%). All 71 duplicate blocks in ST occur within the same file, typically in symmetric getter/setter pairs for multi-channel peripherals (e.g. ADC rank configuration). TI, by contrast, has 76 cross-file clones — code that is structurally identical across different peripheral modules. This pattern arises from TI's UniComm architecture, where `dl_unicommuart.h`, `dl_unicommspi.h`, and `dl_unicommi2cc.h` share a common General Serial Controller (GSC) base but replicate similar interrupt-handling and configuration boilerplate in each specialisation.

The higher duplication in TI is not inherently a quality defect — it reflects a design choice that trades some code reuse for peripheral-specific readability. However, from a maintenance perspective, ST's lower duplication means fewer locations to update when fixing a bug or adapting to silicon errata.

5.2.5 Cyclomatic complexity

Cyclomatic complexity (CC) measures the number of linearly independent paths through a function; higher values indicate more decision branches and, consequently, harder-to-test code [45]. We ran Lizard on every function in both driver sets and summarise the results in Table 5.5.

Table 5.5: Cyclomatic complexity comparison — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Analysed functions	3 542	2 539
Mean CC	4.44	7.35
Median CC	4	5
Maximum CC	49	141
Functions with CC $>$ 10	44 (1.2 %)	335 (13.2 %)
Functions with CC $>$ 20	12 (0.3 %)	97 (3.8 %)

ST's driver code is markedly flatter: the average function has fewer than five decision points, and only 1.2 % of functions exceed the widely accepted threshold of CC = 10 [47]. The maximum (CC = 49) occurs in a temperature-calculation helper (`LL_ADC_CALC_TEMPERATURE`) that contains a long chain of calibration look-ups.

TI's SDK shows a heavier tail: 13.2 % of its functions exceed CC = 10, with a peak of 141 in a single function. Many of these high-complexity functions reside in the UniComm serial-controller layer, where a large `switch` dispatches configuration variants for UART, SPI, and I^2C through a shared code path. While this design consolidates peripheral logic, it concentrates branching in ways that complicate unit testing and formal verification.

From a maintainability standpoint, ST's approach of decomposing peripheral logic into many small, low-complexity inline functions aligns better with safety-coding guidelines that recommend CC $\leq$ 10 per function.

5.2.6 API surface

Table 5.6 compares the API design characteristics.

Table 5.6: API surface comparison — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Public functions	3 543	2 500
Static functions	1	42
Inline functions	3 543	2 032
Functional macros	194	27
Constant macros	3 993	3 271
Enums	1	361
Typedefs	0	470
Avg. parameters/func.	1.17	1.69
Max parameters	8	8
`extern "C"` guards	33	43
Include guards	33	0
Parameter distribution (functions by param count)		
0 parameters	492	120
1 parameter	2 303	1 160
2 parameters	658	933
3 parameters	135	274
4+ parameters	48	139

The two APIs reflect fundamentally different design philosophies:

- **ST LL** exposes 3 543 public functions, *all* of which are `__STATIC_INLINE`. The API is almost entirely composed of one-parameter getters and zero-parameter status queries (65% of functions take 0 or 1 parameter). Type safety relies on constant macros rather than enums. The naming convention follows the `LL_PERIPHERAL_Action` pattern consistently across all 33 files.
- **TI driverlib** exposes 2 500 public functions, of which 2 032 are inline (81%). It makes heavier use of enums (361 enum types) and typedefs (470) for configuration parameters, which improves type safety and IDE autocompletion at the cost of a more complex type graph. The naming convention follows `DL_Peripheral_Action` uniformly. Functions tend to take more parameters on average (1.69 vs. 1.17), reflecting a more configuration-object-oriented style where a single call sets multiple related fields.

ST's 194 functional macros versus TI's 27 show that ST still uses preprocessor computation in places (e.g. register-offset calculations, bit-field assembly) where TI has migrated the same logic into

inline functions. While both approaches compile to equivalent machine code, the inline-function approach provides better debugging and type checking.

5.2.7 MISRA C:2012 static-analysis findings

Both SDKs were analysed using `cppcheck` with the MISRA C:2012 addon [47]. Table 5.7 presents the per-rule breakdown.

Table 5.7: Cppcheck MISRA-addon findings — ST LL vs. TI driverlib.

Rule	**Description**	**ST**	**TI**
17.3	Function declared implicitly	3 541	281
11.4	Conversion between pointer and integer	84	4
10.6	Composite expression assigned to wider type	29	—
20.7	Macro parameter not enclosed in parentheses	21	—
7.3	Lower-case “l” suffix on literal	18	—
10.4	Incompatible essential types in operation	—	67
12.1	Precedence of operators unclear	—	52
18.4	Pointer arithmetic	—	76
15.5	Multiple return statements	4	14
10.1	Operands of inappropriate essential type	—	13
3.1	Comment contains nested comment	—	11
Other	Various minor rules	6	33
Total		**3 703**	**551**

Interpretation

The raw Cppcheck output reports 3 703 findings for ST and 551 for TI. Rule 17.3 accounts for 3 541 ST findings and 281 TI findings. The current analysis attributes many ST findings to processing header-only inline code in isolation, but the original preprocessing inputs and a compilation-database rerun are not available with the report artifacts. We therefore present exclusion of Rule 17.3 only as a sensitivity calculation, not as proof that every such finding is a false positive. Under that exclusion, the counts are 162 for ST and 270 for TI. These are tool-reported findings for the analysed source sets and do not constitute MISRA certification or a complete compliance assessment.

The remaining findings also require source-level review. Rule 11.4 includes casts between integers and peripheral base-address pointers, a common low-level-driver pattern, while TI’s reported findings

span rules concerning pointer arithmetic, essential types, and operator precedence. Counts alone do not establish the severity or correctness impact of these patterns.

Security analysis (CWE)

Both driver layers returned **zero CWE findings** from `cppcheck`'s CERT/CWE checker. Neither SDK contains patterns flagged as common weakness enumerations, which is expected for thin, register-level driver code that performs no dynamic allocation, string handling, or network I/O.

5.3 SDK coverage comparison

Beyond driver-layer quality, the breadth of an SDK — how many peripherals, middleware stacks, and communication protocols it supports out of the box — determines the development effort required for a real product. We mapped the complete contents of both SDKs file by file and identified exclusive features on each side.

5.3.1 Peripheral driver coverage

Table 5.8 lists the peripheral drivers present in only one SDK.[1]

[1]For brevity, only a representative subset of ST's 25 exclusive drivers is shown; the full list is available in the analysis output.

Table 5.8: Exclusive peripheral drivers — features in one SDK only.

Vendor	Driver	Capability
ST-exclusive (25 drivers)		
ST	CORDIC accelerator	HW trigonometric/hyperbolic math
ST	DAC (12-bit)	Full DAC peripheral
ST	Ethernet MAC	On-chip networking
ST	USB Host / Device / DRD	Full USB stack support
ST	I3C controller	Next-gen I3C bus
ST	HASH accelerator (SHA)	HW SHA-1/SHA-256
ST	OPAMP	On-chip operational amplifier
ST	ICACHE controller	Configurable instruction cache
ST	XSPI (Octo/Quad-SPI)	Advanced memory-mapped SPI
ST	LPUART, USART, SmartCard, SMBus	Extended serial protocols
TI-exclusive (16 drivers)		
TI	High-Speed ADC (HSADC)	Dedicated 9.4 MSPS ADC block
TI	General Serial Controller (GSC)	Unified serial engine
TI	UniComm (I2C/SPI/UART)	Configurable serial block
TI	Signal Processing Subsystem	On-chip analog processing
TI	Low-Frequency Subsystem (LFSS)	Unified RTC + tamper + scratchpad
TI	Key Store Controller	HW key management
TI	Error Action Module	HW automatic fault handling
TI	UART extend (IrDA, Manchester)	Extended UART modes

ST leads with 25 exclusive peripheral drivers to TI's 16. The most significant gaps are in connectivity: ST offers USB, Ethernet, and I3C, which are entirely absent from the MSPM33C321A. TI compensates with stronger analog integration (high-speed ADC, signal processing subsystem) and a unified serial architecture that consolidates UART, SPI, and I2C into a single configurable peripheral.

5.3.2 Middleware coverage

Table 5.9 compares the middleware stacks.

Table 5.9: Exclusive middleware — present in one SDK only.

Vendor	Middleware	Purpose
ST	FileX	FAT-compatible embedded file system
ST	LevelX	Flash wear-leveling (NAND/NOR)
ST	LwIP	Lightweight TCP/IP stack
ST	USBX	USB device/host class drivers
ST	mbedTLS	TLS/SSL library (TLS 1.2/1.3)
ST	Open Bootloader	Multi-interface bootloader
ST	stFCF	Security provisioning framework
ST	stCryptoLib	Comprehensive crypto library
TI	Edge AI	On-device ML inference
TI	LVGL	Embedded graphics/GUI library
TI	LIN Bus	Automotive LIN protocol
TI	DALI	Smart lighting protocol
TI	IQmath	Fixed-point DSP math library
TI	Post-Quantum Crypto	ML-DSA (Dilithium) signatures
TI	Curve25519	Modern elliptic curve crypto

ST provides 8 exclusive middleware components focused on connectivity (USB, Ethernet/IP, file system) and security (TLS, crypto library, secure boot). TI provides 7 exclusive components that target emerging domains: Edge AI for on-device machine learning, LVGL for embedded GUIs, automotive protocols (LIN, DALI), and forward-looking cryptography (post-quantum ML-DSA, Curve25519).

5.3.3 Cryptographic algorithm coverage

Table 5.10 provides a detailed comparison of the algorithms available in each vendor's crypto libraries.

Table 5.10: Cryptographic algorithm coverage comparison.

Algorithm	**ST (stCryptoLib)**	**TI (crypto/)**
AES (ECB, CBC, CTR, GCM)	✓	✓
AES-CCM / AES-XTS	✓	—
SHA-224/256	✓	✓
SHA-384/512, SHA-3	✓	—
HMAC	✓	✓
CMAC / KMAC	✓	—
RSA (PKCS#1)	✓	✓
ECDSA / ECDH	✓	✓
EdDSA (Ed25519)	✓	✓
SM2 / SM3 (Chinese std.)	✓	—
ChaCha20-Poly1305	✓	—
ML-DSA (Post-Quantum)	—	✓
Curve25519 / X25519	—	✓
TRNG (HW)	✓	✓
PKA (HW)	✓	✓
Key Store (HW)	✓	✓

ST offers the broader algorithm portfolio (10+ exclusive algorithms), covering legacy standards (SHA-1), Chinese national standards (SM2/SM3), and authenticated-encryption modes (AES-CCM, ChaCha20-Poly1305). TI, while narrower, holds a forward-looking advantage with post-quantum cryptography (ML-DSA/Dilithium) and modern elliptic-curve primitives (Curve25519), which positions it well for long-term security requirements [48].

5.3.4 Communication protocol coverage

Table 5.11 summarises the communication protocol support.

Table 5.11: Communication protocol support comparison.

Protocol	ST	TI
UART / SPI / I2C / I2S / CAN FD	✓	✓
USB (Device + Host)	✓	—
Ethernet	✓	—
I3C	✓	—
SMBus / SmartCard (ISO 7816)	✓	—
LPUART / USART (synchronous)	✓	—
IrDA mode / Manchester encoding	—	✓
LIN Bus	—	✓

ST supports 7 exclusive communication protocols, with USB and Ethernet being the most impactful for connected applications. TI brings 3 exclusive capabilities: IrDA mode, Manchester encoding (both through its extended UART), and LIN Bus for automotive applications.

5.4 Static-analysis summary

Table 5.12 provides a consolidated view of the static-analysis results.

Table 5.12: Static-analysis scoreboard — STM32CubeC5 LL vs. TI MSPM33 driverlib.

Dimension	ST result	TI result	Advantage
Code lines	22 292	27 972	ST (leaner)
Documentation coverage	97.3 %	82.6 %	ST
Code duplication	2.19 %	7.25 %	ST
Cyclomatic complexity	4.44 avg	7.35 avg	ST (flatter)
API functions	3 543	2 500	ST (wider)
Type safety (enums)	1	361	TI
MISRA-addon findings excluding Rule 17.3	162*	270	Sensitivity only
Security (CWE)	0	0	Tie
Exclusive peripherals	25	16	ST
Exclusive middleware	8	7	Balanced
Crypto algorithms	10+ exclusive	2 exclusive	ST (breadth), TI (PQC)
Comm. protocols	7 exclusive	3 exclusive	ST

*Sensitivity calculation; Rule 17.3 findings were not individually adjudicated.

Within the analysed source sets, ST's LL driver layer contains fewer code lines, higher measured documentation coverage, less measured duplication, and a lower adjusted MISRA-addon finding count when Rule 17.3 is excluded as a sensitivity calculation. The SDK inventory also records broader ST coverage in several connectivity and security categories.

TI, however, brings distinct strengths: stronger type safety through enums and typedefs, forward-looking post-quantum cryptography, superior analog integration (9.4 MSPS ADC), a unified serial architecture that simplifies board design, and emerging-domain middleware (Edge AI, LVGL, automotive protocols). These advantages position TI well for mixed-signal and AI-at-the-edge applications where connectivity is less critical than analog performance.

5.5 Functional analysis of the driver layer

Static metrics describe the structure of the two driver libraries, but they do not establish whether equivalent application behaviours can be exercised in practice. We therefore prepared twelve bare-metal scenarios that pair TI driverlib examples with STM32CubeC5 LL examples. The scenarios cover conversion, GPIO control, I^2C, SPI, and UART behaviours, as summarised in Table 5.13. They constitute the first part of the functional phase; the second part will apply the same process to middleware stacks.

5.5.1 Benchmark design and observables

Each scenario starts in `main()`, configures the relevant peripheral through the vendor's low-level API, triggers an observable operation, and verifies completion. The TI examples are drawn from the MSPM33 SDK [34], while the ST counterparts use the STM32CubeC5 LL example set [39]. The examples are bare-metal applications: no RTOS scheduling or middleware service is included in the measured scope. For serial transfers, the controller/master side initiates communication and the target/responder or peripheral/slave side completes the paired exchange. Completion is exposed through a peripheral flag or interrupt, a byte-for-byte buffer comparison, an LED state, a debugger-visible value, or a UART output.

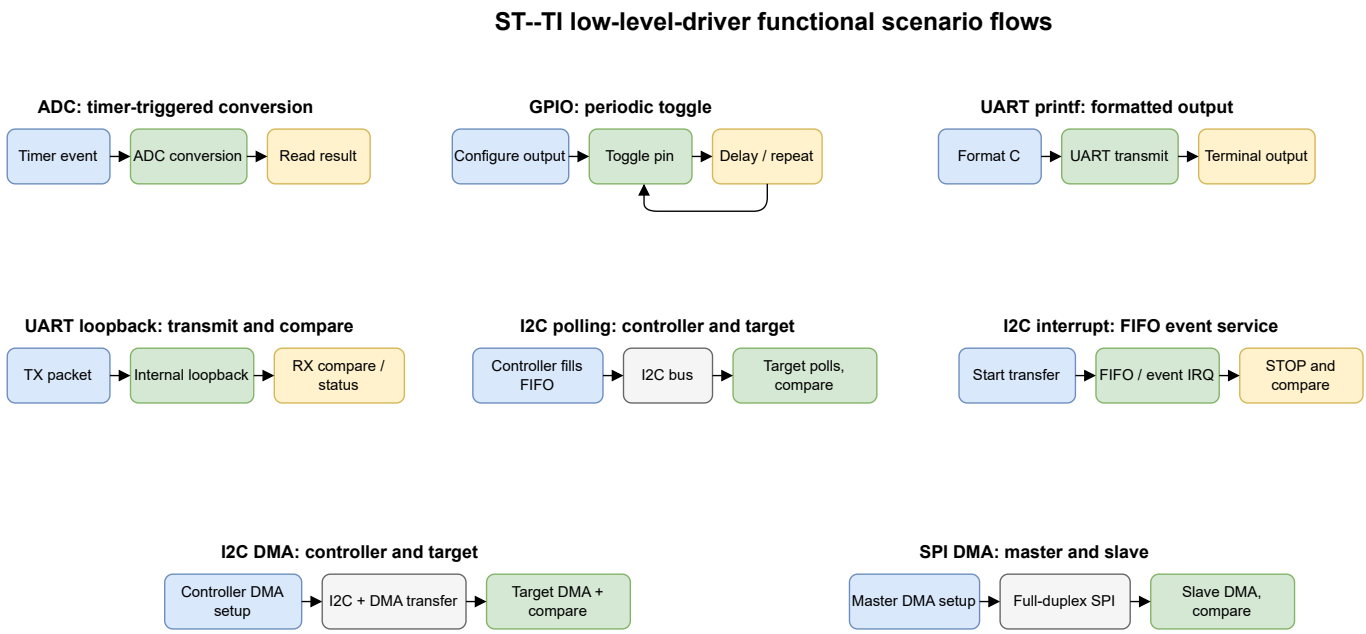

Figure 5.1: Functional flows of the eight low-level-driver scenario families. I^2C and SPI panels represent their paired controller/target and master/slave projects, respectively; the flows define functional behaviour and not a timing window.

Table 5.13: ST–TI low-level-driver functional scenario mapping.

ID	Scenario	TI driverlib example	STM32CubeC5 LL example
1	ADC, timer-triggered conversion	ADC single conversion	ADC single conversion
2	GPIO output toggle	GPIO I/O toggling	GPIO toggle
3	I^2C controller transmit, DMA	I2C two-board DMA, controller	I2C DMA, controller
4	I^2C target receive, DMA	I2C two-board DMA, target	I2C DMA, responder
5	I^2C controller transmit, interrupt	I2C two-board interrupt, controller	I2C interrupt, controller
6	I^2C target receive, interrupt	I2C two-board interrupt, target	I2C interrupt, responder
7	I^2C controller transmit, polling	I2C two-board polling, controller	I2C polling, controller
8	I^2C target receive, polling	I2C two-board polling, target	I2C polling, responder
9	SPI controller/master full-duplex, DMA	SPI two-board DMA, controller	SPI DMA, master
10	SPI peripheral/slave full-duplex, DMA	SPI two-board DMA, peripheral	SPI DMA, slave
11	UART internal loopback	USART loopback	USART loopback
12	UART formatted output	UART printf	USART printf

The I^2C and SPI examples validate the transfer path by comparing received and expected buffers; a successful completion is indicated by the normal LED state, whereas an error enters the example's error indication path. The UART loopback scenario similarly checks two four-byte packets. In the formatted-output scenario, both implementations emit the same single-character payload, `C`. The ADC and GPIO scenarios provide a conversion value or a visible pin state, respectively. Figure 5.1 gives the functional flow for each scenario family. These observables establish a functional pass/fail contract; they are separate from execution-time and energy measurements.

5.5.2 Results and interpretation

The functional projects were rebuilt with IAR EWARM 9.60.3 using high optimisation configured for size on both platforms. We extracted read-only code, read-only data, and read-write data from

the linker maps, whose memory accounting is documented by IAR [15]. We define flash as the sum of read-only code and read-only data, and RAM as read-write data. The latter includes the linker-reserved stack; both implementations reserve 512 bytes per image, preserving a common RAM baseline. The results therefore describe complete linked images, including startup and vector-table costs, rather than only application source files.

Table 5.14 reports the twelve matched pairs. A negative delta means that STM32C5 uses less memory. STM32C5 has the smaller flash image in the ADC scenario, both I^2C DMA roles, both I^2C polling roles, and the two UART scenarios. MSPM33 has the smaller flash image for GPIO, both interrupt-driven I^2C roles, and both SPI DMA roles. STM32C5 uses less RAM in every I^2C and SPI pair, while the small ADC, GPIO, and UART examples are equal in RAM.

Table 5.14: Whole-image footprint results from IAR high/size linker maps (bytes). Flash is RO code plus RO data and RAM is RW data [15]; negative deltas favour STM32C5.

Scenario	MSPM33 flash	STM32 flash	Flash delta	MSPM33 RAM	STM32 RAM	RAM delta
ADC timer trigger	1 476	1 416	-60 (-4.1%)	513	513	0 (0.0%)
GPIO toggle	656	772	+116 (+17.7%)	512	512	0 (0.0%)
I^2C DMA controller	1 928	1 840	-88 (-4.6%)	553	517	-36 (-6.5%)
I^2C DMA responder	1 744	1 687	-57 (-3.3%)	593	556	-37 (-6.2%)
I^2C interrupt controller	1 580	1 732	+152 (+9.6%)	557	514	-43 (-7.7%)
I^2C interrupt responder	1 388	1 460	+72 (+5.2%)	600	553	-47 (-7.8%)
I^2C polling controller	1 420	1 376	-44 (-3.1%)	556	513	-43 (-7.7%)
I^2C polling responder	1 412	1 340	-72 (-5.1%)	608	552	-56 (-9.2%)
SPI DMA controller	1 704	2 151	+447 (+26.2%)	592	556	-36 (-6.1%)
SPI DMA peripheral	1 876	1 940	+64 (+3.4%)	592	552	-40 (-6.8%)
UART printf	1 326	1 318	-8 (-0.6%)	512	512	0 (0.0%)
USART loopback	1 588	1 580	-8 (-0.5%)	532	532	0 (0.0%)
Total, 12 independent images	**18 098**	**18 612**	**+514 (+2.8%)**	**6 720**	**6 382**	**-338 (-5.0%)**

The aggregate must be interpreted as a portfolio sum, not the memory requirement of one deployable firmware image. Across the twelve independently linked projects, MSPM33 has the smaller aggregate flash footprint by 514 bytes (2.8%), whereas STM32C5 has the smaller aggregate RAM allocation by 338 bytes (5.0%). The small 8-byte flash differences in UART printf and loopback are near ties and should not be interpreted as an architectural advantage.

The map attribution explains the mixed flash result. STM32C5 typically carries a larger fixed startup and vector-table contribution: approximately 460 bytes of read-only data against 276 bytes on MSPM33. GPIO makes this effect visible: STM32C5 uses less read-only code (312 versus 380 bytes), yet its complete image is 116 bytes larger because the fixed read-only-data cost dominates the minimal application. In the DMA and polling I^2C scenarios, the STM32 application removes unused buffers, generic initialisation, and inactive state, so its scenario-specific savings exceed this fixed cost. Interrupt-driven I^2C retains event and error handlers, leaving MSPM33 ahead in flash but STM32C5 ahead in RAM.

SPI DMA controller is STM32C5's largest flash loss, at 447 bytes. The result is not explained solely by the vector table: the STM32 application explicitly owns GPIO, DMA, SPI, NVIC, end-of-transfer and DMA-error handling, buffer comparison, LED feedback, and delay behaviour, whereas the TI project resolves more configuration through SysConfig and DriverLib. This is a valid whole-SDK implementation result; it does not establish that STM32 hardware intrinsically requires 447 additional bytes for SPI DMA. Conversely, the smaller STM32 RAM totals arise primarily from fewer persistent buffers and state variables, not from a smaller stack, whose 512-byte reservation is equal on both sides.

These footprint results are independent of runtime performance. No timing, energy, or hardware-success conclusion is inferred from linker maps. In particular, I^2C runtime equivalence still requires evidence of matching address acknowledgement, 37-byte transfer completion, STOP behaviour, target comparison, and identical error handling.

Table 5.15 defines the controls required before publishing runtime results. It separates the established footprint evidence from platform and measurement conditions that must be normalised. This distinction is especially important for the ADC scenario: the TI example stops after a timer-triggered conversion for debugger inspection, while the ST example continuously reads and transmits conversion values. Those reporting behaviours must be removed from the timing boundary or made equivalent.

Table 5.15: Controls required before quantitative ST–TI functional results are compared.

Control	Current observation	Required action
TI target device	The footprint maps target MSPM33C321A, paired with STM32C5A3ZG.	Record the exact device configuration again in every future runtime result.
Core and clock	Footprint does not establish matched runtime clock settings.	Record and normalise core clock, peripheral clock, and timer source for every pair.
Build configuration	Debug artifacts and optimisation-labelled example folders coexist.	Build both sides with documented, matched compiler and linker options.
Peripheral protocol	Payload length, bus speed, ADC input, and sampling settings are not fully matched.	Set identical functional parameters and preserve them in the scenario configuration.
Measurement boundary	LEDs, buttons, UART output, and debugger stops are present in examples.	Measure from the common trigger to verified completion, excluding setup and reporting.
Sampling protocol	No repetition count, warm-up policy, or aggregation is implemented.	Capture repeated samples and report the statistic, dispersion, and pass/fail rate.

Once these controls are resolved, each row of Table 5.13 can produce a directly comparable result set: functional pass/fail status, code and data footprint from matched linker maps, and a normalised timing or energy metric. Until then, the evidence supports the conclusion that the two libraries cover comparable low-level behaviours across the selected scenarios, not that one implementation is faster or smaller.

5.6 FreeRTOS middleware benchmark

The low-level-driver benchmark measures isolated peripheral integrations; it does not exercise the scheduler and kernel services that structure a concurrent embedded application. We therefore extended the functional comparison to FreeRTOS [29] using the same three scenario families introduced in Section 4.3: basic processing, cooperative yielding, and priority-driven preemption. Project manifests identify FreeRTOS Kernel V11.2.0 in the STM32CubeC5 projects and V11.1.0 in the MSPM33

SDK 1.04.00.00 projects; the vendor packages provide the corresponding source baselines [30, 34, 39]. This version difference is retained as a fairness limitation.

5.6.1 Scenario design and configuration

All scenarios use the native FreeRTOS API, a 1 kHz tick, preemption enabled, time slicing disabled, dynamic allocation, ten priority levels, and an 8 KiB FreeRTOS heap. Worker stack depth is 128 FreeRTOS stack units and the reporter depth is 512 units. Every reporter delays for 30 s before capturing the cumulative counters and transmitting the observation through a 115200-baud UART. Figure 5.2 summarises the task topology.

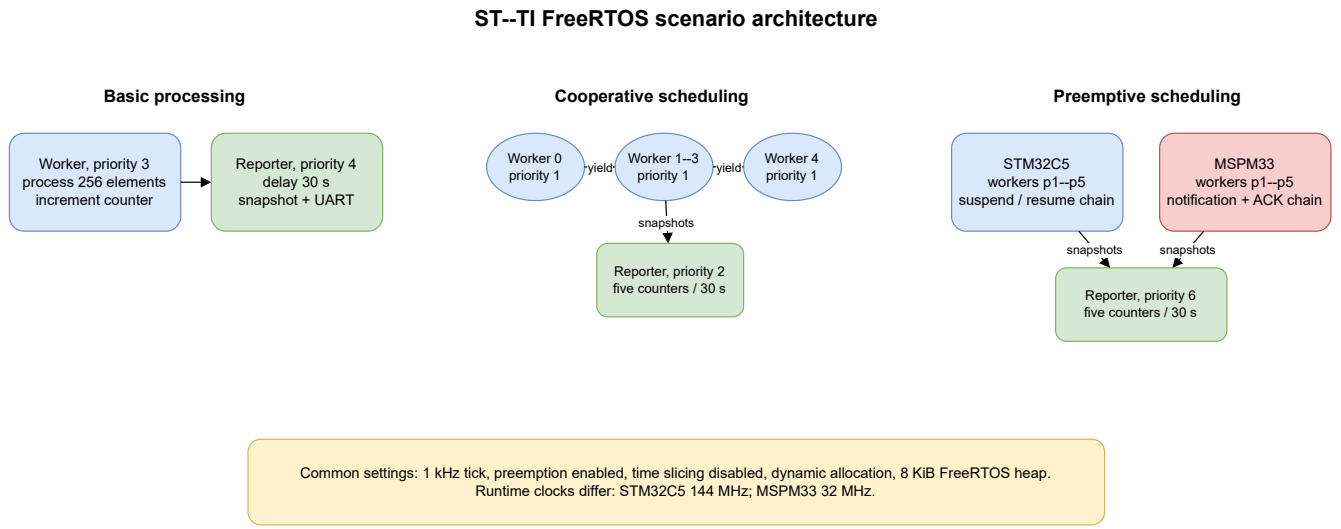

Figure 5.2: Task architecture of the three ST–TI FreeRTOS scenarios. The preemptive scenario deliberately shows the non-equivalent synchronisation paths found in the measured projects.

Table 5.16: FreeRTOS scenario semantics and reported metric.

Scenario	Task behaviour	Reported metric
Basic	One priority-3 worker transforms a 256-element volatile array and increments a counter; a priority-4 reporter samples it.	Completed processing iterations/s
Cooperative	Five priority-1 workers call `taskYIELD()` and increment separate counters; a priority-2 reporter samples all five.	Mean iterations/thread/s
Preemptive	Five workers use priorities 1–5 and a priority-6 reporter. STM32 uses a suspend/resume chain; TI uses task notifications and an acknowledgement.	Mean completed chains/thread/s

The basic and cooperative scenarios preserve the same worker count, creation order, priorities, and counter semantics on both platforms. The preemptive projects do not preserve the same synchronisation primitive: STM32 uses `vTaskSuspend()` and `vTaskResume()`, while TI uses direct task notifications plus a final acknowledgement. Its result therefore compares the two complete implementations but cannot isolate the cost of a common preemption mechanism.

5.6.2 Measurement method and fairness

Runtime values were obtained from UART captures supplied after executing both boards. Each row in Table 5.17 is the mean of the available 30-second windows: three STM32 and two TI windows for basic processing, two and four for cooperative processing, and two and six for preemptive processing. No warm-up interval or dispersion statistic was recorded. Cooperative and preemptive reporters advance a nominal time by 30 s; because UART output occurs before the next delay, reporting time is outside their denominator.

The largest fairness difference is the configured core clock: STM32C5 runs at 144 MHz with instruction cache enabled, whereas MSPM33 runs at 32 MHz. Both targets use a Cortex-M33 and Virtual Floating-Point version 5 (VFPv5) single-precision configuration, but the clock ratio is 4.5:1. We therefore report the raw rates and a clock-normalised ratio

$$R_{\text{norm}} = \frac{R_{\text{STM32}}/144}{R_{\text{MSPM33}}/32}.$$

A value near one indicates that the raw difference is explained approximately by configured clock frequency; it is not a direct measurement of cycles per context switch.

5.6.3 Results

Table 5.17: Observed FreeRTOS runtime rates and clock-normalised STM32/MSPM33 ratio.

Scenario	**STM32C5 rate**	**MSPM33 rate**	$\boldsymbol{R_{\text{norm}}}$
Basic processing	43 172.93 iterations/s	9 570.47 iterations/s	1.002
Cooperative	226 652.76 iterations/thread/s	35 783.68 iterations/thread/s	1.408
Preemptive	55 015.52 chains/thread/s	7 110.96 chains/thread/s	1.719

Basic processing scales almost exactly with the configured clock ratio: after normalisation, STM32C5 and MSPM33 differ by only 0.2%. The cooperative observation retains a normalised ratio of 1.408, but this value includes the complete worker loop, FreeRTOS port, compiler runtime, cache and memory behaviour; it does not measure `taskYIELD()` in isolation. The preemptive ratio is larger

still, but the different kernel versions and synchronisation primitives prevent attribution to scheduler efficiency. One TI cooperative run printed `ERROR` because the shared integer-truncated balance check allowed only a one-count deviation; the individual counters remained within the observed scheduling pattern, so this is treated as a benchmark-check limitation rather than a demonstrated kernel failure.

All six images were rebuilt with IAR EWARM 9.60.3 using high optimisation for size (`CCOptLevel=3`, `CCOptStrategy=1`). Flash is read-only code plus read-only data; RAM is read-write data, following the linker-map accounting used in Section 5.5.2 [15].

Table 5.18: FreeRTOS whole-image footprint under IAR high/size optimisation (bytes).

	Flash			RAM		
Scenario	**STM32**	**TI**	**ST–TI**	**STM32**	**TI**	**ST–TI**
Basic	9 052	12 854	-3 802 (-29.6%)	10 196	10 140	+56 (+0.6%)
Cooperative	9 640	13 430	-3 790 (-28.2%)	9 188	9 132	+56 (+0.6%)
Preemptive	10 012	13 894	-3 882 (-27.9%)	9 188	9 112	+76 (+0.8%)
Total	**28 704**	**40 178**	**-11 474 (-28.6%)**	**28 572**	**28 384**	**+188 (+0.7%)**

Unlike the low-level-driver portfolio, the middleware result consistently favours STM32C5 in flash: its three images are 27.9–29.6% smaller, for an aggregate reduction of 11 474 bytes. TI uses slightly less RAM in every scenario, but the difference is only 56–76 bytes per image. These totals include each platform's complete startup, configuration, C runtime, FreeRTOS kernel/port, application, heap, and stack reservations. TI links an externally built FreeRTOS library, while STM32 compiles the kernel sources in each project; configuration also differs in task notifications, stack-overflow checking, application task tags, and thread-local-storage pointers. The footprint gap is consequently a result for these deployable SDK configurations, not a kernel-only size comparison.

Appendix C.3 preserves the raw UART windows, project configuration, primitive mapping, and linker-map totals used for these calculations.

5.6.4 Takeaway

The FreeRTOS benchmark adds two findings to the ST–TI comparison. First, the STM32C5 projects produce substantially smaller linked flash images while MSPM33 retains a marginal RAM advantage. Second, raw runtime rates cannot be read as a direct platform ranking: the basic scenario is almost entirely explained by the 4.5:1 clock ratio, and the more complex scenarios retain differences that are confounded by kernel configuration and, for preemption, different synchronisation semantics. A controlled follow-up should use equal clocks, one FreeRTOS version and configuration, equal primitives,

equal sample counts, a warm-up policy, and cycle-level measurement boundaries.

Conclusion

This chapter presented the ST-versus-TI comparison of the STM32CubeC5 LL layer and TI driverlib. The static phase covered seven quality dimensions and a complete SDK coverage mapping. The functional phase then mapped twelve bare-metal scenarios spanning ADC, GPIO, I^2C, SPI, and UART behaviour, and documented their observable completion criteria.

Within the analysed source sets, STM32CubeC5 records higher measured documentation coverage, lower duplication and average cyclomatic complexity, and a lower adjusted Cppcheck MISRA-addon finding count. The SDK inventory also identifies broader ST coverage in several connectivity and security categories. These findings describe the selected SDK versions and analysis configuration; they do not establish overall ecosystem maturity, certified MISRA compliance, runtime performance, or product-level software quality.

TI's analysed driverlib makes wider use of enums and typedefs, while the SDK inventory identifies distinct analog, post-quantum cryptography, AI, graphics, and automotive capabilities. These are source-structure, datasheet, and inventory observations rather than measured advantages in execution time, analog performance, safety, or application suitability.

The functional phase provides whole-image footprint evidence for both the low-level-driver and FreeRTOS portfolios. The driver scenarios slightly favour MSPM33 in aggregate flash and STM32C5 in RAM, whereas all three FreeRTOS images favour STM32C5 in flash and MSPM33 by a small RAM margin. Runtime observations are available for FreeRTOS, but unequal clocks, kernel versions, configuration options, sample counts, and preemptive primitives limit their interpretation. The basic result is almost exactly clock-proportional; cooperative and preemptive differences require a controlled follow-up before they can support scheduler-efficiency conclusions.

General Conclusion and Perspectives

This project addressed the need for comparative evidence that separates measurable software-stack behaviour from assumptions about vendor ecosystems. We defined a scenario-based method in which ST serves as the reference, equivalent observable behaviour is established before comparison, build and measurement conditions are recorded, and footprint or runtime results are interpreted at the relevant architectural layer. MCU Workflow Studio and the MCU Benchmark Copilot Agent support this method by structuring linker-map analysis, project preparation, provenance tracking, fairness review, and semantic-equivalence checks. Their contribution lies in formalising and assisting the workflow; the validity of each benchmark still depends on its source artifacts, build records, and hardware evidence.

The completed ST–GD32 comparison provided footprint and reported runtime observations for five USB and FreeRTOS scenarios. It revealed a recurring trade-off rather than a uniform advantage for either ecosystem. STM32 used more flash, with much of the classified difference associated with HAL components such as RCC, but required substantially less RAM in the USB scenarios. It also showed a lower reported USB initialisation time, while steady-state USB and FreeRTOS observations remained close. These results support targeted investigation of RCC footprint and finer USB feature selection, but they do not establish that the same pattern applies to other devices, compiler settings, or software stacks.

The ST–TI work provided a different type of evidence. Static analysis characterised the two low-level driver sets, while twelve matched bare-metal projects produced whole-image flash and RAM measurements. The aggregate result slightly favoured MSPM33 in flash and STM32C5 in RAM, with scenario-level outcomes varying according to startup costs, retained handlers, buffers, and application-owned configuration. The comparison therefore cannot be reduced to a single ecosystem ranking. The FreeRTOS extension produced the opposite flash pattern: STM32C5 used 27.9–29.6 % less flash in all three scenarios, while MSPM33 retained a 56–76-byte RAM advantage. Runtime observations were collected, but the 144 MHz versus 32 MHz clock difference explains nearly all of the basic-processing gap. Different kernel versions, configuration options, and preemptive primitives limit the cooperative and preemptive interpretation, so the current evidence does not establish a scheduler-efficiency ranking.

Several limitations define the scope of these findings. The evaluated boards, SDK versions, scenarios, IAR configuration, and linker-map accounting rules form part of each result and limit its generalisation. Static-analysis findings depend on the selected source sets and tool configurations, while heuristic classifications or quality indicators are not substitutes for expert review. Aggregate footprint totals across independent projects are portfolio summaries, not the memory requirement of one deployable application. Finally, the supporting tools were designed to improve consistency and traceability, but this report does not contain a controlled productivity study or a complete cross-environment reproducibility assessment.

Future work should first repeat the ST–TI runtime phase under the controls identified in Chapter 5. Each scenario should use equal documented clocks, the same FreeRTOS version and configuration, equivalent synchronisation primitives, a common measurement interval, equal sample counts, a warm-up policy, and reported dispersion. Energy measurements may then be added using the same functional boundaries. The benchmark set can subsequently be extended to additional middleware and vendor platforms, provided that each new comparison preserves the same evidence and fairness requirements.

The supporting tools also require further validation. MCU Workflow Studio should be tested against a larger labelled set of linker maps, with mapping accuracy and repeatability reported separately, and its parser may be extended beyond IAR EWARM. The MCU Benchmark Copilot Agent should be evaluated through recorded campaigns that distinguish successful automation, manual intervention, unresolved unknowns, and hardware-dependent steps. These extensions would broaden the evidence base without weakening the central rule established by this work: no comparative conclusion should exceed the measurements and controls that support it.

Appendix A

Glossary

Architectural layer: A classification of firmware symbols into one of four categories — application, middleware, low-level driver, or system runtime — used to attribute memory costs to comparable functional blocks across vendors.

Board Support Package (BSP): A set of drivers and configuration files that adapt a vendor's generic HAL to a specific evaluation or development board.

Context switch: The process by which an RTOS saves the CPU state of the currently running task and restores the state of the next task to be executed. Context-switch efficiency directly affects multi-threaded throughput.

Cross-vendor benchmark: A controlled experiment in which the same workload runs on two vendor platforms under identical conditions (clock, optimisation level, scenario logic), producing comparable metrics.

Firmware footprint: The amount of non-volatile (flash/ROM) and volatile (RAM) memory a firmware image occupies once compiled and linked. It is a static measurement taken from the linked binary, not a dynamic runtime observation.

Linker map: A text file produced by the linker at the end of the build process. It lists every module and function linked into the final image together with its code size, constant-data size, and variable-data size. It is the primary input to the footprint analysis tool.

Middleware: Software libraries that sit between the hardware abstraction layer and the application, providing protocol or service logic (e.g. USB device stack, FreeRTOS kernel, FatFS, LwIP).

Semantic equivalence: The property that two benchmark implementations — one on the reference platform and one on the competitor — exhibit identical observable behaviour (task structure, timing, synchronisation, reporting format) so that their comparison is fair.

Software stack: A vertical slice of firmware functionality (e.g. USB device, FreeRTOS integration) comprising application code, middleware, and hardware drivers, benchmarked as a unit.

Tick (RTOS): The periodic timer interrupt that drives the RTOS scheduler. A 1 ms tick means the scheduler evaluates task readiness every millisecond.

Appendix B

Acronyms

AES	Advanced Encryption Standard
AI	Artificial Intelligence
API	Application Programming Interface
BSP	Board Support Package
CAN	Controller Area Network
CDC	Communication Device Class
CLI	Command-Line Interface
CMSIS	Cortex Microcontroller Software Interface Standard
CRC	Cyclic Redundancy Check
CPU	Central Processing Unit
CSV	Comma-Separated Values
CWE	Common Weakness Enumeration
DAC	Digital-to-Analog Converter
DMA	Direct Memory Access
DRD	Dual-Role Device (USB)
DSP	Digital Signal Processing
DWT	Data Watchpoint and Trace
ECC	Error Correction Code
ECDSA	Elliptic Curve Digital Signature Algorithm
EWARM	Embedded Workbench for ARM
FDCAN	Flexible Data-rate CAN
FPU	Floating-Point Unit
FS	Full-Speed (USB)
FSP	Flexible Software Package (Renesas)

FST	Faculty of Sciences of Tunis
GPIO	General-Purpose Input/Output
GSC	General Serial Controller
GUI	Graphical User Interface
HAL	Hardware Abstraction Layer
HID	Human Interface Device
HS	High-Speed (USB)
HSADC	High-Speed Analog-to-Digital Converter
HSI	High-Speed Internal oscillator
I3C	Improved Inter-Integrated Circuit
IAR	IAR Systems (toolchain vendor)
IDE	Integrated Development Environment
INPT	Institut National des Postes et Télécommunications
IoT	Internet of Things
IRQ	Interrupt Request
JSON	JavaScript Object Notation
KB	Kilobyte
LFSS	Low-Frequency Subsystem
LIN	Local Interconnect Network
LL	Low-Layer (driver)
LLM	Large Language Model
LOC	Lines of Code
MB	Megabyte
MCD	Microcontrollers Division
MCU	Microcontroller Unit
MISRA	Motor Industry Software Reliability Association
ML-DSA	Module-Lattice Digital Signature Algorithm
MPU	Memory Protection Unit
MSC	Mass Storage Class
MSPS	Mega Samples Per Second
MW	Middleware
NXP	NXP Semiconductors
OTG	On-The-Go (USB)
PDF	Portable Document Format
PFE	Projet de Fin d'Études

PKA	Public Key Accelerator
RAM	Random-Access Memory
RCC	Reset and Clock Control
ROM	Read-Only Memory (flash)
RTOS	Real-Time Operating System
SDK	Software Development Kit
SQL	Structured Query Language
SRAM	Static Random-Access Memory
ST	STMicroelectronics
SWO	Serial Wire Output
TI	Texas Instruments
TLS	Transport Layer Security
TRNG	True Random Number Generator
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
UTM	University of Tunis El Manar
VFP	Vector Floating-Point
VS Code	Visual Studio Code
XLSX	Office Open XML Spreadsheet
YAML	YAML Ain't Markup Language

Appendix C

Complementary Code Listings

This appendix provides representative code excerpts that complement the tool and benchmark descriptions in the main chapters. They illustrate the formats consumed and produced by our tools and the firmware structure of the benchmark scenarios.

C.1 IAR EWARM linker map excerpt

Listing C.1 shows an abbreviated IAR EWARM linker map for a USB HID project. This is the format parsed by MCU Workflow Studio (Chapter 2). Each entry lists a module, its read-only code size, read-only data, and read-write data — the three quantities from which flash and RAM footprint are derived.

```
*******************************************************************************
*** MODULE SUMMARY
***

    Module                ro code  ro data  rw data
    ------                -------  -------  -------
    main.o                    164       --       --
    stm32u5xx_hal.o           212       --        4
    stm32u5xx_hal_gpio.o      456       --       --
    stm32u5xx_hal_rcc.o     4 560       16       --
    stm32u5xx_hal_pcd.o     2 496       --       --
    stm32u5xx_ll_usb.o      2 492       --       --
    usbd_core.o             1 048       --       --
    usbd_ctlreq.o             876       --       --
    usbd_hid.o                620       --       --
    usbd_conf.o               524       --       32
    usbd_desc.o               461      268       --
    system_stm32u5xx.o        128       16        4
```

```
    startup_stm32u575xx.o    394       512        --
    ------                -------   -------   -------
    Total:                14 431       812        40

*******************************************************************************
*** ENTRY LIST
***

    Entry                  Address    Size  Type  Object
    -----                  -------    ----  ----  ------
    HAL_Init              0x0800'1234   52  Code  stm32u5xx_hal.o
    HAL_RCC_OscConfig     0x0800'2000  1024  Code  stm32u5xx_hal_rcc.o
    HAL_RCC_ClockConfig   0x0800'2400   896  Code  stm32u5xx_hal_rcc.o
    USBD_Init             0x0800'3000   128  Code  usbd_core.o
    USBD_HID_Init         0x0800'3200    96  Code  usbd_hid.o
    ...
```

Listing C.1: Abbreviated IAR EWARM linker map (USB HID, STM32U575).

C.2 ST–TI functional footprint reproducibility record

This appendix records the build and accounting conditions behind the ST-versus-TI functional footprint results in Chapter 5. The TI projects are MSPM33 SDK driverlib examples [34]; their counterparts are STM32CubeC5 LL examples [39]. All twelve pairs were rebuilt with IAR EWARM 9.60.3 using high optimisation for size (`CCOptLevel=3`, `CCOptStrategy=1`). The IAR linker map reports read-only code, read-only data, and read-write data [15]. We use the following whole-image accounting:

$$\text{Flash} = \text{RO code} + \text{RO data}, \qquad \text{RAM} = \text{RW data}.$$

Both projects in every pair reserve a 512-byte stack in the linker map. The table is therefore reproducible from the matching IAR map files and includes the deployable image's startup and vector-table cost. It is a portfolio of independent images, not the footprint of one firmware that combines all scenarios.

Table C.1: Reproducibility record for the ST–MSPM33 high/size functional footprint study (bytes).

Scenario	MSPM33 flash	STM32 flash	Delta	MSPM33 RAM	STM32 RAM	Delta
ADC timer trigger	1 476	1 416	-60	513	513	0
GPIO toggle	656	772	+116	512	512	0
I²C DMA controller	1 928	1 840	-88	553	517	-36
I²C DMA responder	1 744	1 687	-57	593	556	-37
I²C interrupt controller	1 580	1 732	+152	557	514	-43
I²C interrupt responder	1 388	1 460	+72	600	553	-47
I²C polling controller	1 420	1 376	-44	556	513	-43
I²C polling responder	1 412	1 340	-72	608	552	-56
SPI DMA controller	1 704	2 151	+447	592	556	-36
SPI DMA peripheral	1 876	1 940	+64	592	552	-40
UART printf	1 326	1 318	-8	512	512	0
USART loopback	1 588	1 580	-8	532	532	0
Total, 12 independent images	**18 098**	**18 612**	**+514**	**6 720**	**6 382**	**-338**

Negative deltas favour STM32C5. The corresponding interpretation, fairness limits, and scenario-flow diagrams are presented in Section 5.5; this appendix is intentionally limited to reproducibility facts and raw whole-image totals.

C.3 ST–TI FreeRTOS reproducibility record

The FreeRTOS middleware benchmark in Section 5.6 uses three matched project pairs from STM32CubeC5 and MSPM33 SDK 1.04.00.00 [34, 39]. All six images were built with IAR EWARM 9.60.3 at high optimisation for size (`CCOptLevel=3`, `CCOptStrategy=1`) [15]. The common configuration includes a 1 kHz tick, preemption enabled, time slicing disabled, dynamic allocation, ten priorities, and an 8 KiB FreeRTOS heap. STM32C5 was configured at 144 MHz and MSPM33 at 32 MHz; these are source configuration values rather than independent clock measurements. Project manifests identify FreeRTOS Kernel V11.2.0 for STM32C5 and V11.1.0 for MSPM33. In the preemptive scenario, STM32 uses `vTaskSuspend()`/`vTaskResume()`, whereas MSPM33 uses direct task notifications and a final acknowledgement.

Table C.2: Raw UART observation windows for the ST–TI FreeRTOS benchmark.

Scenario	**Platform**	**Recorded 30-second deltas**
Basic	STM32C5	1 295 358; 1 295 330; 1 295 351. Elapsed: 30 000; 30 005; 30 006 ms.
Basic	MSPM33	287 141; 287 135. Elapsed: 30 000; 30 005 ms.
Cooperative	STM32C5	33 998 170; 33 997 657 total worker iterations
Cooperative	MSPM33	5 367 603; 5 367 557; 5 367 559; 5 367 490 total worker iterations
Preemptive	STM32C5	8 252 379; 8 252 277 total completed chains
Preemptive	MSPM33	1 066 661; 1 066 639; 1 066 639; 1 066 637; 1 066 635; 1 066 655 total completed chains

Table C.3: IAR high/size map totals for the ST–TI FreeRTOS projects (bytes).

Scenario	**STM32 flash**	**MSPM33 flash**	**STM32 RAM**	**MSPM33 RAM**
Basic	9 052	12 854	10 196	10 140
Cooperative	9 640	13 430	9 188	9 132
Preemptive	10 012	13 894	9 188	9 112
Total	**28 704**	**40 178**	**28 572**	**28 384**

The runtime record is not a controlled equal-clock experiment. Sample counts differ, no warm-up policy is documented, and the preemptive implementations use different synchronisation primitives. The tables preserve the raw evidence so that later equal-clock measurements can be compared without replacing or silently reinterpreting the original observations.

C.4 Agent configuration file

Listing C.2 shows the configuration of the Creation-Porting specialist agent from the MCU Benchmark Copilot Agent system (Chapter 3). The YAML frontmatter declares metadata, description, argument hints, and tool permissions; the Markdown body contains the behavioural instructions, autonomy contract, and supported workflow directions.

```
---
name: Creation-Porting
description: "Create benchmark projects from scratch or port existing
  benchmarks in any direction (STM32->competitor, competitor->STM32,
  vendor->vendor, board->board). Use when: generating target project
  files, creating from a scenario description, separating
  portable/RTOS/board logic, or acquiring missing reference firmware."
argument-hint: A benchmark requirement, scenario description, scenario
  name, firmware name, or a reference project plus target platform.
tools: ['read', 'search', 'edit', 'vscode', 'todo', 'web', 'execute']
---

# Creation-Porting Agent

## Supported Directions
- From scratch (scenario description only)
- STM32 -> competitor
- Competitor -> STM32
- Vendor A -> Vendor B
- Board A -> Board B
- Name only (resolve, acquire, create)

## Autonomy Contract
- Operates autonomously for file generation, project creation, and
  material acquisition (web fetches from official sources).
- Asks only before flashing/programming/resetting hardware.
- Acquires missing firmware, CMSIS, BSP, startup, linker from
  official vendor GitHub repos and SDKs without confirmation.

## IAR Template Resolution
Before creating any project:
1. Look up target board in config/iar_templates.yaml
2. Follow resolution: exact board -> family fallback -> type fallback
3. Acquire template if status == "needs_download"
4. Use materialised template as read-only project baseline
```

```
5. Place benchmark code through app_integration_points only

## Multi-Layer Separation
1. Portable application layer (writable) - benchmark logic
2. RTOS / middleware layer (protected) - select, don't modify
3. Board-specific layer (protected) - select existing vendor files
4. Build surface (configure only) - .ewp references, defines, paths

## Semantic Preservation Rule
Preserve semantics and observable behaviors, not vendor API names.
Map CMSIS-RTOS2 <-> native FreeRTOS <-> ThreadX as needed via
app-level adapter files. Never modify vendor HAL source.
```

Listing C.2: Creation-Porting agent configuration (abbreviated).

C.5 FreeRTOS cooperative benchmark scenario

Listing C.3 shows the core task logic of the cooperative processing benchmark scenario from Chapter 4. Five equal-priority threads each increment a counter and yield; a reporter thread prints the results every 30 seconds.

```
/* Shared counters for the five worker threads */
volatile uint32_t counters[5] = {0};

/* Worker task: increment own counter, then yield */
void WorkerTask(void *param)
{
    uint32_t id = (uint32_t)param;
    for (;;)
    {
        counters[id]++;
        taskYIELD();
    }
}

/* Reporter task: print all counters every 30 s */
void ReporterTask(void *param)
{
    (void)param;
    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(30000));
        printf("Counters: %lu %lu %lu %lu %lu\r\n",
               counters[0], counters[1], counters[2],
               counters[3], counters[4]);
        /* Reset for next interval */
        for (int i = 0; i < 5; i++)
            counters[i] = 0;
    }
}

/* Task creation (in main, after HAL and kernel init) */
void CreateBenchmarkTasks(void)
{
    for (uint32_t i = 0; i < 5; i++)
    {
        xTaskCreate(WorkerTask, "Worker", 128,
```

```
37                      (void *)i, tskIDLE_PRIORITY + 1, NULL);
38      }
39      xTaskCreate(ReporterTask, "Reporter", 256,
40                  NULL, tskIDLE_PRIORITY + 2, NULL);
41      vTaskStartScheduler();
42  }
```

Listing C.3: FreeRTOS cooperative scenario — task functions (STM32, simplified).

C.6 Footprint tool CLI output

Listing C.4 shows a representative terminal output from a footprint benchmark run using the MCU Workflow Studio command-line interface (Chapter 2).

```
$ mcu-workflow-cli benchmark \
    --source stm32_usb_hid.map --source-vendor stm32 \
    --target gd32_usb_hid.map  --target-vendor gd32 \
    --out results/

[INFO] Parsing source map: stm32_usb_hid.map (IAR EWARM)
[INFO] Parsing target map: gd32_usb_hid.map (IAR EWARM)
[INFO] Classifying 47 source symbols into layers...
[INFO] Classifying 31 target symbols into layers...
[INFO] Mapping engine: 42 pairs scored, 38 accepted (confidence >= 0.72)
[INFO] 4 pairs queued for review (confidence 0.55-0.72)

=== FOOTPRINT COMPARISON (USB HID) ===

Layer           | STM32 ROM | GD32 ROM  | Diff
----------------|-----------|-----------|--------
Application     |   2 580 B |   1 942 B | +33%
HAL drivers     |  10 426 B |     820 B | +1171%
MW stack        |   2 859 B |   5 743 B | -50%
----------------|-----------|-----------|--------
TOTAL ROM       |  16 269 B |   9 175 B | +77%
TOTAL RAM       |   3 446 B |   9 608 B | -65%

Verdict: STM32 uses +77% more ROM but -65% less RAM.
Largest classified STM32 modules:
    HAL RCC:       4560 B
    HAL/LL USB:    4988 B
Causal attribution: unresolved without complete maps and classification
   diagnostics.

[INFO] Results written to results/benchmark_results.xlsx
[INFO] Mapping diagnostics: results/mapping_diagnostics.json
[INFO] Review queue (4 items): results/review_queue.csv
```

Listing C.4: MCU Workflow Studio CLI — benchmark summary output.

C.7 Static-analysis automation script (ST vs. TI)

Listing C.5 shows an abbreviated version of the Python script used to extract API surface and documentation coverage metrics for the ST-versus-TI static analysis (Chapter 5). The full script analyses duplication, documentation, and API design in a single pass over the driver source trees.

```
"""
Firmware benchmark analysis script:
- Code duplication detection (sliding-window hash)
- Documentation coverage (Doxygen tag extraction)
- API surface area analysis (functions, macros, enums)
"""
import os, re, json, hashlib
from collections import defaultdict, Counter
from pathlib import Path

TI_DIR = r"mspm33_sdk/source/ti/driverlib"
ST_DIR = r"STM32CubeC5/stm32c5xx_drivers/ll"

def get_c_files(directory):
    files = []
    for root, _, filenames in os.walk(directory):
        for f in filenames:
            if f.endswith(('.c', '.h')):
                files.append(os.path.join(root, f))
    return files

def analyze_api(files):
    """Extract API surface metrics from C source files."""
    result = {
        'public_functions': 0, 'static_functions': 0,
        'inline_functions': 0, 'macros_functional': 0,
        'macros_constant': 0, 'enums': 0, 'typedefs': 0,
        'param_distribution': Counter(),
    }
    for filepath in files:
        with open(filepath, 'r', errors='ignore') as f:
            content = f.read()
        # Count public functions (not static)
        pub = re.findall(
            r'(?<!static\s)(?:__STATIC_INLINE\s+|inline\s+)?'
            r'(?:void|bool|uint\w+|int\w+|DL_\w+|LL_\w+)\s+'
```

```
            r'(\w+)\s*\(([^)]*)\)', content)
        for name, params in pub:
            result['public_functions'] += 1
            n = len([p for p in params.split(',')
                     if p.strip()]) if params.strip() else 0
            result['param_distribution'][n] += 1
        # Count enums, typedefs, macros
        result['enums'] += len(re.findall(
            r'\benum\s+\w*\s*\{', content))
        result['typedefs'] += len(re.findall(
            r'\btypedef\s+', content))
        result['macros_functional'] += len(re.findall(
            r'#define\s+\w+\([^)]*\)', content))
        result['macros_constant'] += len(re.findall(
            r'#define\s+\w+\s+[^(]', content))
    return result

if __name__ == '__main__':
    for label, path in [("TI", TI_DIR), ("ST", ST_DIR)]:
        files = get_c_files(path)
        api = analyze_api(files)
        print(f"\n=== {label} API Surface ===")
        for k, v in api.items():
            print(f"  {k}: {v}")
```

Listing C.5: Abbreviated `analyze_firmware.py` — API and documentation extraction.

Bibliography

[1] STMicroelectronics, *About ST*. [Online]. Available: https://www.st.com/content/st_com/en/about/st_company_information.html. Accessed: 2026.

[2] STMicroelectronics, *2024 Annual Report and Financial Highlights*. [Online]. Available: https://investors.st.com/. Accessed: 2026.

[3] STMicroelectronics, *Strategy and Innovation*. [Online]. Available: https://www.st.com/content/st_com/en/about/innovation---technology/innovation---technology.html. Accessed: 2026.

[4] STMicroelectronics, *STM32 32-bit Arm Cortex MCUs*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html. Accessed: 2026.

[5] Arm Ltd., *Cortex-M Series Processors*. [Online]. Available: https://developer.arm.com/Processors/Cortex-M. Accessed: 2026.

[6] STMicroelectronics, *STM32Cube Ecosystem*. [Online]. Available: https://www.st.com/en/ecosystems/stm32cube.html. Accessed: 2026.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[8] Python Software Foundation, *The Python Language Reference, version 3.11*. [Online]. Available: https://docs.python.org/3.11/. Accessed: 2026.

[9] The Qt Company, *Qt for Python (PySide6) Documentation*. [Online]. Available: https://doc.qt.io/qtforpython/. Accessed: 2026.

[10] The pandas development team, *pandas: Powerful Python Data Analysis Toolkit*. [Online]. Available: https://pandas.pydata.org/. Accessed: 2026.

[11] E. Gazoni and C. Clark, *openpyxl: A Python Library to Read and Write Excel 2010 xlsx/xlsm Files*. [Online]. Available: https://openpyxl.readthedocs.io/. Accessed: 2026.

[12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/.

[13] SQLite Development Team, *SQLite Documentation*. [Online]. Available: https://www.sqlite.org/docs.html. Accessed: 2026.

[14] PyInstaller Development Team, *PyInstaller Manual*. [Online]. Available: https://pyinstaller.org/. Accessed: 2026.

[15] IAR Systems, *IAR Embedded Workbench for Arm — C/C++ Development Guide (ILINK Linker and Linker Map File)*. [Online]. Available: https://www.iar.com/embedded-development-tools/iar-embedded-workbench. Accessed: 2026.

[16] GitHub, *GitHub Copilot Documentation*. [Online]. Available: https://docs.github.com/en/copilot. Accessed: 2026.

[17] Anthropic, *Claude API Documentation*. [Online]. Available: https://docs.anthropic.com/. Accessed: 2026.

[18] OpenAI, *OpenAI API Reference*. [Online]. Available: https://platform.openai.com/docs/. Accessed: 2026.

[19] Ollama, *Ollama: Run Large Language Models Locally*. [Online]. Available: https://ollama.com/. Accessed: 2026.

[20] GitHub, *GitHub Models Documentation*. [Online]. Available: https://docs.github.com/en/github-models. Accessed: 2026.

[21] NXP Semiconductors, *MCUXpresso SDK Documentation*. [Online]. Available: https://mcuxpresso.nxp.com/. Accessed: 2026.

[22] Microsoft, *Visual Studio Code Documentation*. [Online]. Available: https://code.visualstudio.com/docs. Accessed: 2026.

[23] Microsoft, *GitHub Copilot Agent Mode in VS Code*. [Online]. Available: https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode. Accessed: 2026.

[24] Microsoft, *Custom Chat Agents and Skills — VS Code Documentation*. [Online]. Available: https://code.visualstudio.com/docs/copilot/copilot-customization. Accessed: 2026.

[25] IAR Systems, *C-SPY Debugging Guide — Batch Mode (cspybat)*. [Online]. Available: https://www.iar.com/knowledge/support/technical-notes/debugger/. Accessed: 2026.

[26] GigaDevice Semiconductor, *GD32 Microcontrollers*. [Online]. Available: https://www.gigadevice.com/microcontroller. Accessed: 2026.

[27] GigaDevice Semiconductor, *GD32E507xx Datasheet*, Rev. 1.2, 2022. [Online]. Available: https://www.gigadevice.com/microcontroller/gd32e507zet6. Accessed: 2026.

[28] STMicroelectronics, *STM32U575/585 Datasheet — Ultra-low-power with FPU Arm Cortex-M33 MCU*. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32u575zi.pdf. Accessed: 2026.

[29] Amazon Web Services, *FreeRTOS — Real-Time Operating System for Microcontrollers*. [Online]. Available: https://www.freertos.org/. Accessed: 2026.

[30] Amazon Web Services, *FreeRTOS V11.2.0 Release Notes*. [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Kernel/releases/tag/V11.2.0. Accessed: 2026.

[31] Texas Instruments, *About TI*. [Online]. Available: https://www.ti.com/about-ti/company/company-overview.html. Accessed: 2026.

[32] Texas Instruments, *MSPM33C321A Product Page*. [Online]. Available: https://www.ti.com/product/MSPM33C321A. Accessed: 2026.

[33] Texas Instruments, *MSPM33C321A Datasheet*. [Online]. Available: https://www.ti.com/document-viewer/MSPM33C321A/datasheet. Accessed: 2026.

[34] Texas Instruments, *MSPM33-SDK — Software Development Kit for MSPM33*. [Online]. Available: https://www.ti.com/tool/MSPM33-SDK. Accessed: 2026.

[35] Texas Instruments, *LP-MSPM33C321A LaunchPad Development Kit*. [Online]. Available: https://www.ti.com/tool/LP-MSPM33C321A. Accessed: 2026.

[36] Texas Instruments, *SysConfig — System Configuration Tool*. [Online]. Available: https://www.ti.com/tool/SYSCONFIG. Accessed: 2026.

[37] Texas Instruments, *Code Composer Studio IDE*. [Online]. Available: https://www.ti.com/tool/CCSTUDIO. Accessed: 2026.

[38] Texas Instruments, *TI Arm Clang Compiler Tools*. [Online]. Available: https://www.ti.com/tool/ARM-CGT. Accessed: 2026.

[39] STMicroelectronics, *STM32CubeC5 — STM32Cube MCU Package for STM32C5 Series*. [Online]. Available: https://www.st.com/en/embedded-software/stm32cubec5.html. Accessed: 2026.

[40] STMicroelectronics, *STM32C5A3ZG Product Page*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32c5a3zg.html. Accessed: 2026.

[41] STMicroelectronics, *NUCLEO-C5A3ZG Evaluation Board*. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-c5a3zg.html. Accessed: 2026.

[42] STMicroelectronics, *STM32C5 Series Overview*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32c5-series.html. Accessed: 2026.

[43] A. Danial, *cloc — Count Lines of Code*, version 2.08. [Online]. Available: https://github.com/AlDanial/cloc. Accessed: 2026.

[44] D. Marjamäki *et al.*, *Cppcheck — A Static Analysis Tool for C/C++ Code*. [Online]. Available: https://cppcheck.sourceforge.io/. Accessed: 2026.

[45] T. Yin, *Lizard — A Code Complexity Analyser*. [Online]. Available: https://github.com/terryyin/lizard. Accessed: 2026.

[46] K. Kravets, *jscpd — Copy/Paste Detector for Source Code*. [Online]. Available: https://github.com/kucherenko/jscpd. Accessed: 2026.

[47] MISRA, *MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems*, 3rd ed. Nuneaton, UK: MIRA Ltd., 2019.

[48] National Institute of Standards and Technology, *Post-Quantum Cryptography Standardization*. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography. Accessed: 2026.

[49] Arm Ltd., *TrustZone for Armv8-M*. [Online]. Available: https://developer.arm.com/Architectures/TrustZone-for-Armv8-M. Accessed: 2026.

[50] Texas Instruments, *EnergyTrace Technology*. [Online]. Available: https://www.ti.com/tool/ENERGYTRACE. Accessed: 2026.

[51] D. van Heesch, *Doxygen — Generate Documentation from Annotated C/C++ Sources*. [Online]. Available: https://www.doxygen.nl/. Accessed: 2026.

[52] LVGL, *Light and Versatile Graphics Library*. [Online]. Available: https://lvgl.io/. Accessed: 2026.

[53] Arm Ltd., *Mbed TLS — An Open-Source SSL Library*. [Online]. Available: https://www.trustedfirmware.org/projects/mbed-tls/. Accessed: 2026.

[54] Swedish Institute of Computer Science, *lwIP — A Lightweight TCP/IP Stack*. [Online]. Available: https://savannah.nongnu.org/projects/lwip/. Accessed: 2026.

[55] Texas Instruments, *Edge AI for Embedded Processors*. [Online]. Available: https://www.ti.com/technologies/edge-ai.html. Accessed: 2026.